Summary Notes:
AWS Networking and Security

1. Introduction

Here's an overview of what was covered in the session:

1. Networking in Cloud

- Overview
- Significance of Deployment Location

2. Amazon VPC, Subnets, Route Tables and Gateways

- Detailed explanation of Amazon VPC
- Overview of IP Addressing
- Understanding subnets and their configuration
- Overview of route tables and their role in networking

3. Network Security: Security Groups and NACLs

- Explanation of security groups and how they control inbound and outbound traffic
- Understanding Network Access Control Lists (NACLs) and their use cases
- Differences and best practices for using security groups and NACLs

4. Hands-on Lab/Demo: Setting up a Secure VPC

- Step-by-step demonstration of setting up a VPC
- Configuring subnets and route tables
- Implementing security groups

5. Introduction to Serverless Computing

- Definition and benefits of serverless computing.
- Key concepts and operational model.

6. AWS Lambda Basics

- Overview of AWS Lambda.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- Key features and supported programming languages.

7. Integrating AWS Lambda with Other AWS Services

- Common integrations with AWS services such as Amazon S3, Amazon DynamoDB, Amazon API Gateway, Amazon SNS and AWS CloudWatch.
- Use cases and examples of service integrations.

8. Hands-on Lab/Demo: Creating and Deploying Your First Lambda Function

- Creating a Lambda function
- Setting up a trigger
- Configuring the Lambda function
- Testing the function

2. Networking in Cloud

2.1 Introduction to Cloud Networking

Cloud networking offers the ability to connect to systems across cloud environments, similar to on-premise networking. It leverages virtualised infrastructure provided by cloud service providers like AWS, enabling more scalable, flexible and cost-effective network management.

2.2 Transition from Traditional to Cloud Networking

As businesses increasingly move to the cloud, traditional networking methods are being complemented and often replaced by cloud networking solutions. AWS (Amazon Web Services) offers a broad range of networking services that help organisations design, deploy and manage their network infrastructure in the cloud. These services are essential for connecting cloud resources securely, ensuring optimal performance and maintaining high availability. AWS networking services provide the foundation for integrating and managing various applications and workloads in a scalable and secure manner.

2.3 Significance of Deployment Location

When accessing a website or application, data and requests need to travel from the user's device to the server hosting the service and back again. This process typically involves a combination of different transmission mediums:

- Initially, data may travel over Wi-Fi to the user's home router.
- From there, it reaches the Internet Service Provider (ISP) via fibre, cable, ADSL or 5G.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- The ISP then connects to a larger network and the data may travel through undersea fibre cables.

The speed of light dictates the fastest possible transmission speed through these cables. However, the actual speed is affected by the reflection of light within the cable, resulting in a longer travelled distance than the cable's physical length. This inherent delay, combined with network congestion at various points along the route, can significantly impact the response time of websites or applications.

2.4 Advantages of Cloud Technology in Reducing Latency

Cloud technology offers a solution to the latency issues associated with traditional data transmission. By expanding to new geographic regions and deploying applications globally, cloud providers like AWS can significantly reduce latency. AWS's extensive infrastructure allows developers to deploy applications in multiple physical locations around the world quickly and efficiently.

With AWS, developers can position their applications closer to end-users, minimising the distance data must travel and thereby reducing latency. This proximity improves the overall user experience by ensuring faster and more reliable access to websites and applications. Through a few simple steps, applications can be deployed in various global locations, providing a seamless and efficient user experience.

By deploying applications closer to end-users, developers ensure that data does not have to travel long distances. This proximity leads to:

- Faster response times
- Lower latency, and
- Enhanced reliability

Cloud networking significantly improves scalability, flexibility and cost-efficiency. The strategic deployment of applications across AWS's global infrastructure enhances performance and user experience.

3. Amazon VPC

3.1 Overview of Amazon VPC

Amazon Virtual Private Cloud (VPC) is a service that enables the provisioning of a logically isolated section of the AWS Cloud where users can launch and manage AWS resources within a defined virtual network. It provides full control over the virtual networking environment, including IP address range selection, subnet creation and the configuration of route tables and

© Copyright upGrad Education Pvt. Ltd. All rights reserved

network gateways. VPC supports both IPv4 and IPv6, ensuring secure and flexible access to resources and applications.

Before exploring the features of Amazon Virtual Private Cloud (VPC), you need to understand some terminologies that form the foundation of VPC's networking capabilities.

3.2 Key Terminologies

- **Subnets**: A subnet is a range of IP addresses within your VPC. Each subnet resides in a single Availability Zone. Subnets are essential for deploying AWS resources within your VPC.
- **IP addressing**: IP addressing in Amazon VPC allows assigning IPv4 and IPv6 addresses to VPCs and subnets. You can bring your own public IPv4 and IPv6 Global Unicast Addresses (GUAs) to AWS and allocate them to resources in your VPC, such as EC2 instances, NAT gateways and Network Load Balancers. This flexibility ensures secure and efficient access to resources and applications within your VPC.
- **Routing**: Route tables are used to determine the direction of network traffic from your subnet or gateway. They are crucial for managing the flow of traffic within and outside your VPC.
- **Gateways and endpoints**: A gateway connects your VPC to another network. For example, an internet gateway connects your VPC to the internet, whereas a VPC endpoint allows private connectivity to AWS services without using an internet gateway or NAT device.
- **Peering connections**: VPC peering connections enable routing traffic between resources in two VPCs, facilitating resource sharing and network integration.
- **Traffic monitoring**: Traffic monitoring involves copying network traffic from network interfaces, sending it to security and monitoring appliances for deep packet inspection, ensuring enhanced security and network management.
- **Transit gateways**: A transit gateway acts as a central hub for routing traffic among your VPCs, VPN connections and AWS Direct Connect connections, simplifying network architecture and management.
- **VPC flow logs**: Flow logs capture information about the IP traffic going to and from network interfaces in your VPC, providing valuable data for monitoring and troubleshooting.
- **VPN connections**: The AWS Virtual Private Network (AWS VPN) allows you to connect your VPCs to your on-premises networks, ensuring secure and reliable connectivity.

3.3 Features of Amazon VPC

- **Complete Network Control**
  Amazon VPC customises IP address ranges, creates subnets, configures route tables and sets up network gateways.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- **Security**
  Implements multiple layers of security, including security groups and network access control lists (NACLs). It also creates private subnets for backend systems with no internet access and establishes public subnets for web servers with internet access.
- **Scalability and flexibility**
  VPC spans an entire AWS region and subnets can be allocated within specific Availability Zones for optimal resource distribution.
- **Hybrid Networking**
  It establishes a hardware virtual private network (VPN) connection between corporate data centres and the VPC, allowing seamless integration and communication as if they were on the same network.
- **Dual Stack Support**
  VPC provides support for both IPv4 and IPv6 addresses to facilitate modern and scalable network architectures.
- **Compliance and Governance**
  It meets regulatory requirements with robust security and compliance features and maintains governance with detailed access control and monitoring.

3.4 IP Addressing

IP addressing in Amazon VPC provides the flexibility to assign both IPv4 and IPv6 addresses to your VPCs and subnets, ensuring seamless connectivity and communication between resources. IPv4 is the most commonly used IP address format, consisting of four sets of numbers separated by periods (e.g., 192.168.0.1). It supports approximately 4.3 billion unique addresses. IPv6, designed to address the limitations of IPv4, uses the 128-bit address format, significantly expanding the number of available addresses (e.g., 2001:0db8:85a3:0000:0000:8a2e:0370:7334).

In Amazon VPC, you can also use Classless Inter-Domain Routing (CIDR) blocks to define IP address ranges for your VPCs and subnets. A CIDR block allows for flexible subnetting and efficient IP address allocation by specifying a network prefix (e.g., 10.0.0.0/16). Additionally, AWS allows you to bring your own public IPv4 and IPv6 Global Unicast Addresses (GUAs) and allocate them to resources such as EC2 instances, NAT gateways and Network Load Balancers, providing enhanced connectivity and control over your network environment.

3.5 Understanding Subnets and Their Configuration

Subnets in Amazon VPC are used to segment a VPC into smaller, manageable sections, each defined by a range of IP addresses. A subnet resides within a single Availability Zone, providing isolation and security for different types of resources. By creating subnets, you can place public-facing resources, such as web servers, in public subnets with internet access, while

© Copyright upGrad Education Pvt. Ltd. All rights reserved

keeping backend systems, such as databases and application servers, in private subnets without direct internet access.

3.5.1 Features of Subnets:

- **Optimised resource allocation**: Manage IP address ranges efficiently within Availability Zones and allocate resources based on specific application requirements.
- **Enhanced security**: Control the traffic flow with subnet-specific security groups and Network Access Control Lists (NACLs).
- **Scalability**: Easily add or modify subnets as application needs evolve.
- **High availability**: Support fault tolerance and high availability by distributing subnets across multiple Availability Zones.
- **Improved network performance**: Reduce latency by placing resources in appropriate subnets for optimal network performance.

4. Network Security: Security Groups and NACLs

You also learned that security groups act as virtual firewalls for your Amazon EC2 instances, controlling inbound and outbound traffic at the instance level. When you launch an instance in a VPC, you can assign one or more security groups to it. Each security group filters traffic based on the rules you define. These are as follows:

- **Inbound rules**: Control the incoming traffic to the instances. You can specify the allowed protocols, ports and source IP ranges for the inbound traffic.
- **Outbound rules**: Control the outgoing traffic from the instances. Similar to inbound rules, you can define protocols, ports and destination IP ranges for the outbound traffic.

By default, security groups allow all outbound traffic and block all inbound traffic. However, you can customise these rules to meet the specific security requirements of your applications. Security groups operate at the instance level and changes to security group rules are applied immediately, ensuring dynamic and flexible control over network traffic.

4.1 Features of Security Groups

- **Stateful filtering**: If you allow an inbound request from a specific IP address, the response traffic to that IP address is automatically allowed, even if no outbound rule explicitly permits it.
- **Dynamic configuration**: Rules can be added, modified or removed on the fly without restarting the instances.
- **Granular control**: Define rules for specific IP addresses, ranges or other instances within your VPC.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

4.2 Network Access Control Lists (NACLs)

You learned that Network Access Control Lists (NACLs) are another layer of security in Amazon VPC, functioning at the subnet level to control traffic in and out of one or more subnets. Unlike security groups, which are stateful, NACLs are stateless, meaning that return traffic must be explicitly allowed through corresponding inbound and outbound rules.

4.2.1 Features of NACLs:

- **Stateless filtering**: Each request and its corresponding response are evaluated independently, requiring explicit rules for both inbound and outbound traffic.
- **Rule order**: NACLs evaluate rules in order, starting with the lowest numbered rule and proceed until a matching rule is found. If no rule matches, the default action is to deny the traffic.
- **Subnet-level security**: NACLs provide an additional layer of security by applying rules to all traffic entering and leaving a subnet, affecting all instances within that subnet.

4.2.2 Use Cases for NACLs:

- **Additional security layer**: Enhance security using NACLs in conjunction with security groups to provide multiple layers of defence.
- **Broad access control**: Implement broad traffic control policies that apply to all instances in a subnet, such as denying all traffic from a known malicious IP range.
- **Default deny**: Use NACLs to enforce a default deny policy, ensuring that only explicitly allowed traffic can enter or leave a subnet.

4.3 Security Groups and NACLs: Differences and Best Practices

You also explored the differences between security groups and NACLs and best practices for using them:

4.3.1 Differences

Security Groups	NACLs
Operate at the instance level, providing control over inbound and outbound traffic for individual instances	Operate at the subnet level, controlling traffic for all instances within a subnet
Stateful, meaning that if an inbound request is allowed, the outbound response is automatically	Stateless, requiring explicit rules for both inbound and outbound traffic

© Copyright upGrad Education Pvt. Ltd. All rights reserved

allowed	
All rules are evaluated before determining if traffic is allowed	Rules are evaluated in order, starting from the lowest numbered rule to the highest, stopping at the first match
By default, allow all outbound traffic and deny all inbound traffic	By default, allow all inbound and outbound traffic unless otherwise specified

4.3.2 Best Practices for Using Security Groups

- **Least privilege principle**: Only allow the necessary traffic to reach your instances. Define specific IP ranges, ports and protocols.
- **Organise by function**: Group instances with similar functions into the same security groups to streamline rule management.
- **Regular audits**: Periodically review and update security group rules to ensure they meet current security requirements.

4.3.3 Best Practices for Using NACLs

- **Default deny rule**: Implement a final rule to deny all traffic that isn't explicitly allowed, ensuring any unspecified traffic is blocked.
- **Explicit rules for both directions**: Since NACLs are stateless, ensure that you have corresponding rules for inbound and outbound traffic.
- **Order of rules**: Carefully manage the order of NACL rules, as they are evaluated sequentially from lowest to highest number.

4.3.4 Combined Best Practices:

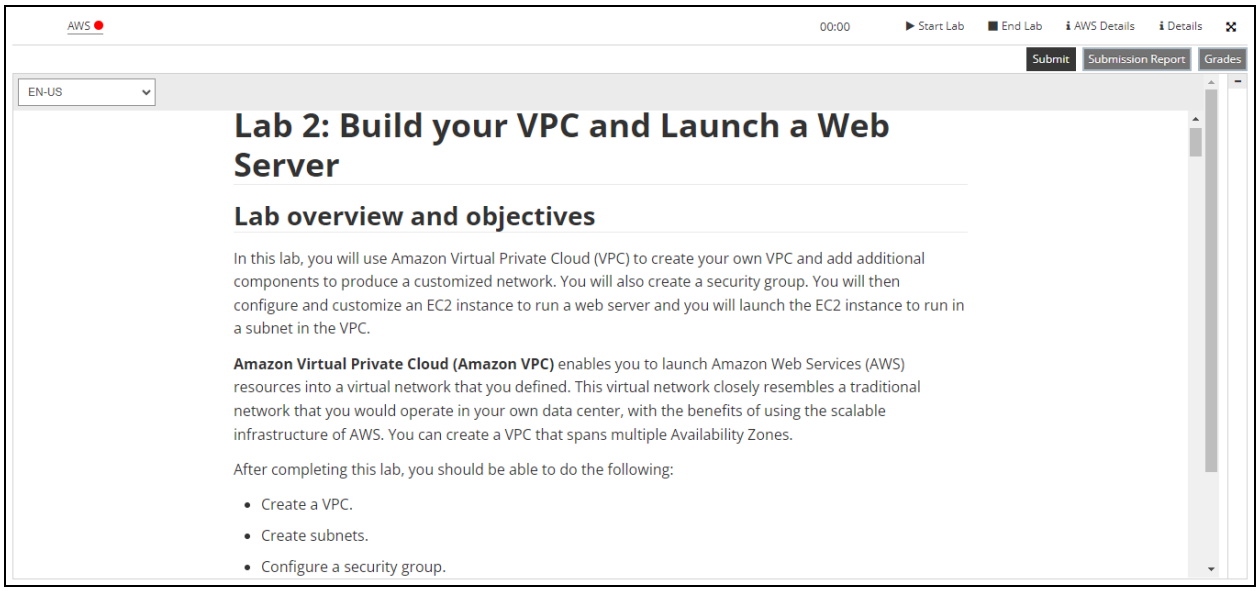
- **Layered security**: Use security groups and NACLs to provide multiple layers of defence. Security groups handle instance-level traffic, whereas NACLs provide subnet-level protection.
- **Monitoring and logging**: Enable VPC flow logs to monitor traffic and identify potential security issues. Regularly review logs to ensure compliance with security policies.
- **Consistency and documentation**: Maintain consistent naming conventions and thoroughly document your security rules and configurations for easy management and troubleshooting.

5. Hands-on Lab/Demo: Setting up a Secure VPC

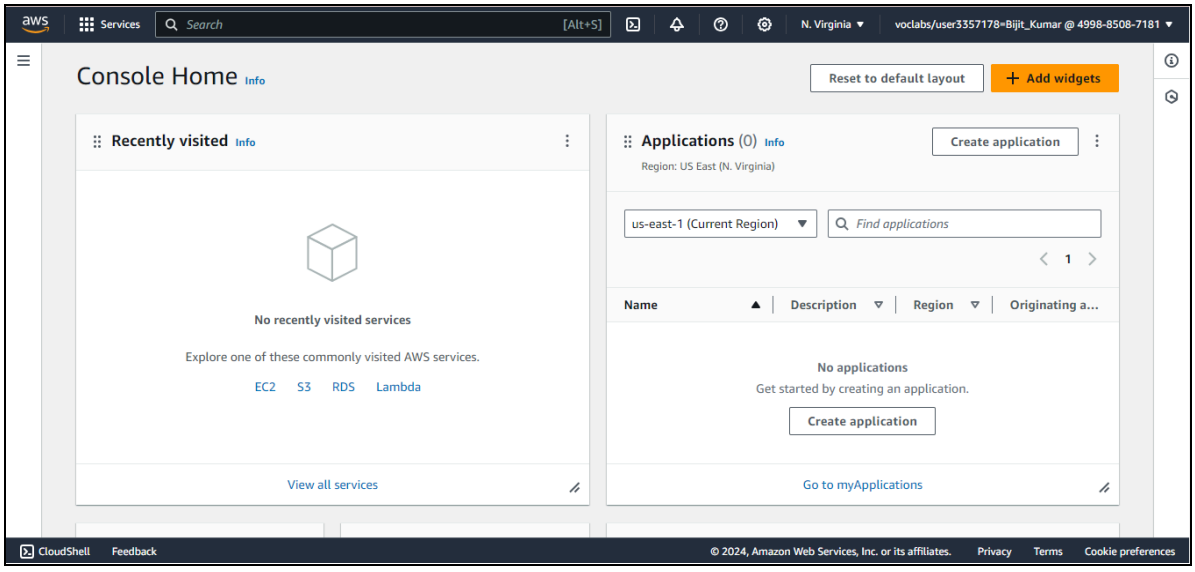
1. At the top of these instructions, choose **Start Lab**.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- The lab session starts.
- A timer displays at the top of the page and shows the time remaining in the session.
  Tip: To refresh the session length at any time, choose **Start Lab** again before the timer reaches 0:00.
- Before you continue, wait until the circle icon to the right of the AWS link in the upper-left corner turns green.



2. To access the AWS Management Console, click on the **AWS** link located in the top left corner. This will open a new tab in your browser, directing you to the console.
   Tip: If a new tab does not appear, your browser may be blocking pop-ups. Typically, a message or icon indicating this will appear at the top of your browser. Click on this message or icon, and select **Allow pop-ups** to proceed.



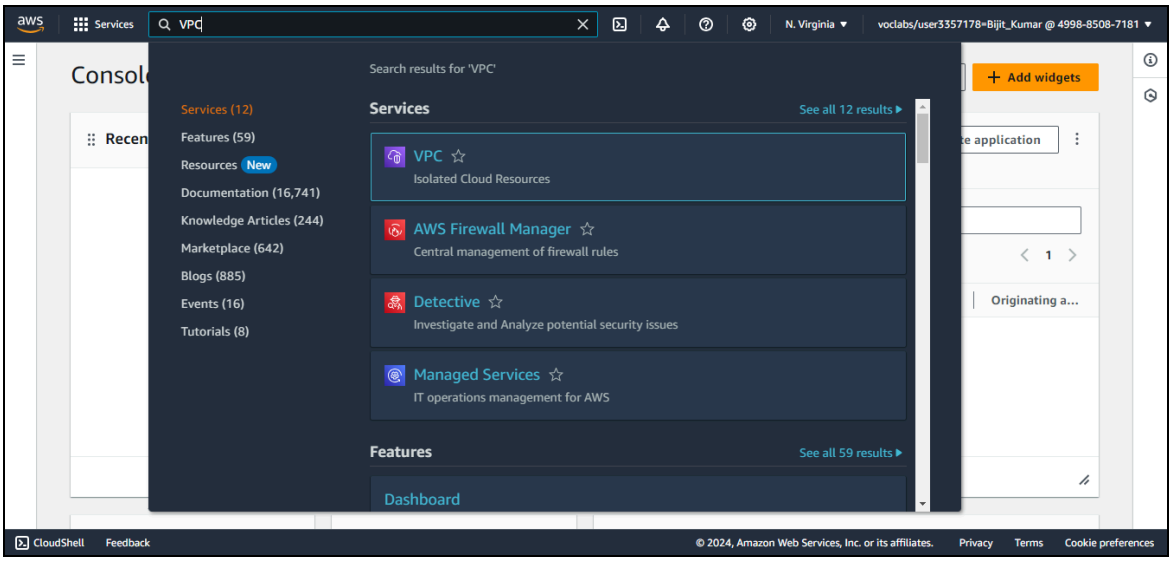
© Copyright upGrad Education Pvt. Ltd. All rights reserved

3. For convenience, arrange the AWS Management Console tab next to this set of instructions. By viewing both tabs simultaneously, you can easily follow the steps of the lab.

Task 1: Create Your VPC

In this task, you will use the VPC and more options in the VPC console to create multiple resources, including a VPC, an Internet Gateway, a public subnet and a private subnet in a single Availability Zone, two route tables and a NAT Gateway.

4. Locate the search box near the **Services** label, and enter **VPC** into it. Select **VPC** from the search results to proceed.



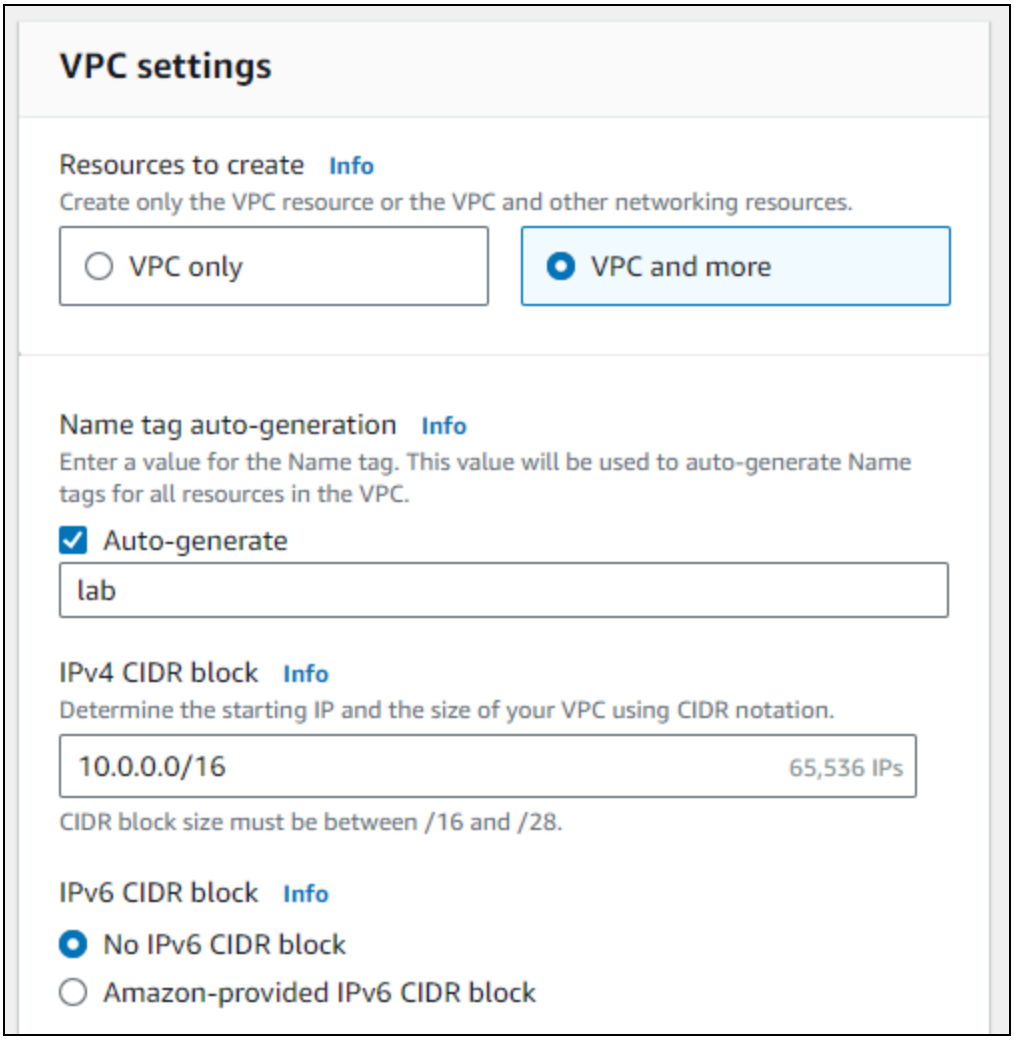
5. Begin creating a VPC.
   - In the top right of the screen, verify that **N. Virginia (us-east-1)** is the region.
   - Next, choose **Create VPC**.
     Note: If you do not see a button with that name, choose the **Launch VPC Wizard** button instead.



6. Configure the VPC details in the **VPC settings** panel on the left:
   - Choose **VPC and more**.

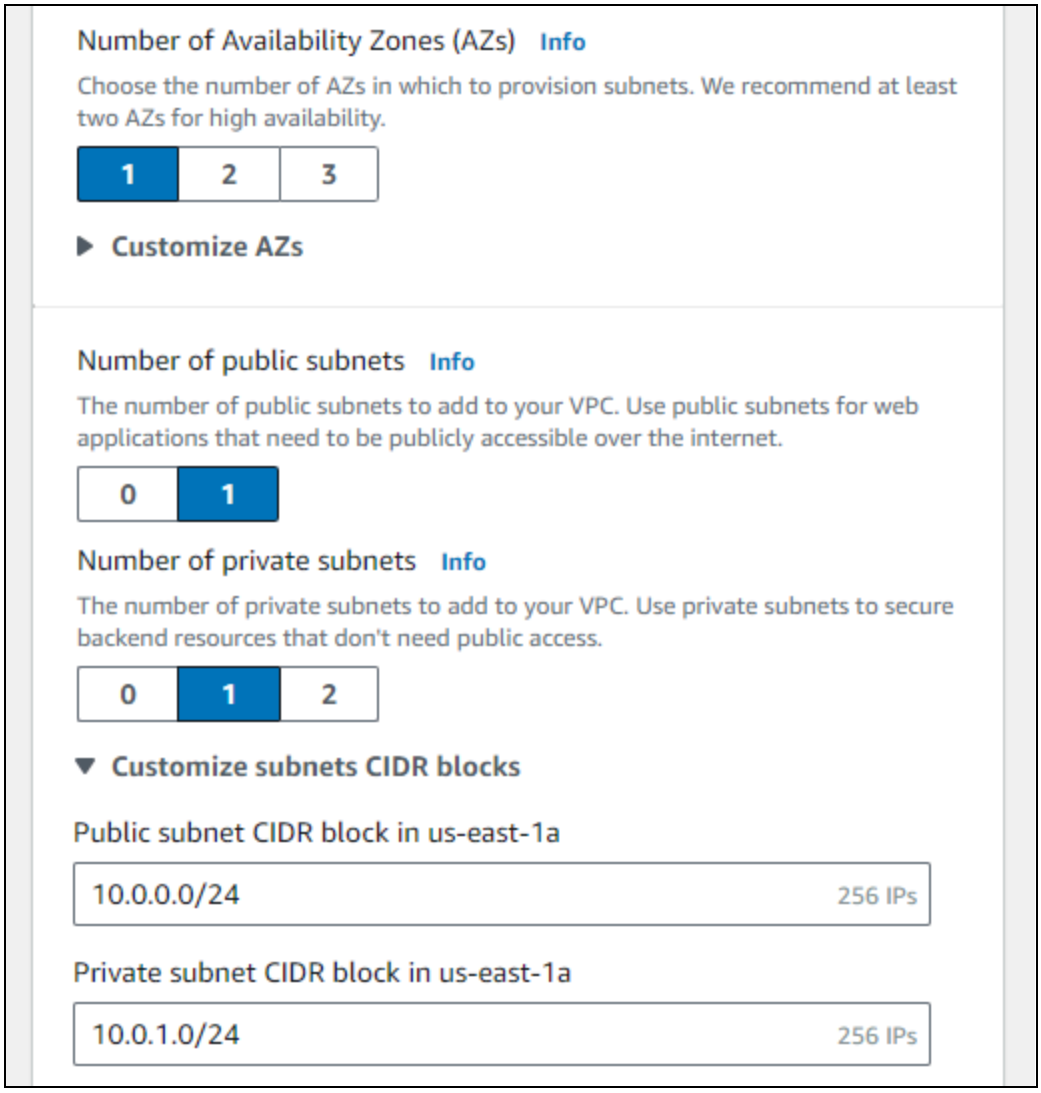
© Copyright upGrad Education Pvt. Ltd. All rights reserved

- Under **Name tag auto-generation**, keep Auto-generate selected, however change the value from project to **`lab`**.
- Keep the **IPv4 CIDR block** set to 10.0.0.0/16
- Keep the **IPv6 CIDR block** selected to **No IPv6 CIDR block**



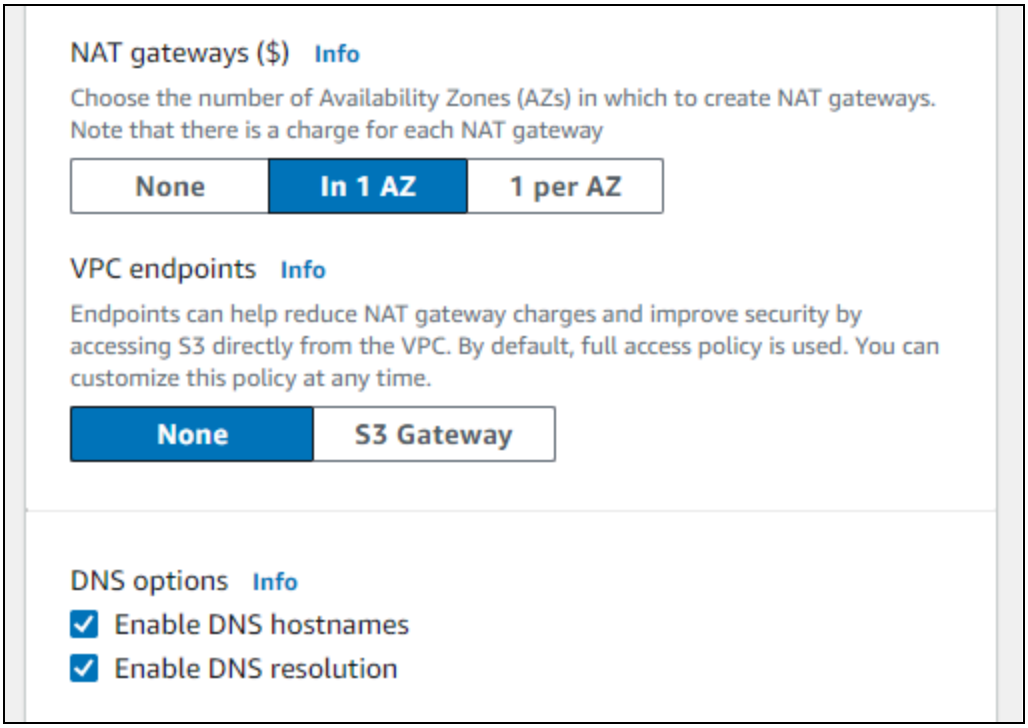
- Keep **Tenancy** be set to **Default**
- For **Number of Availability Zones**, choose **1**.
- For **Number of public subnets**, keep the **1** setting.
- For **Number of private subnets**, keep the **1** setting.
- Expand the **Customize subnets CIDR blocks** section
  - Change **Public subnet CIDR block in us-east-1a** to 10.0.0.0/24
  - Change **Private subnet CIDR block in us-east-1a** to 10.0.1.0/24

© Copyright upGrad Education Pvt. Ltd. All rights reserved

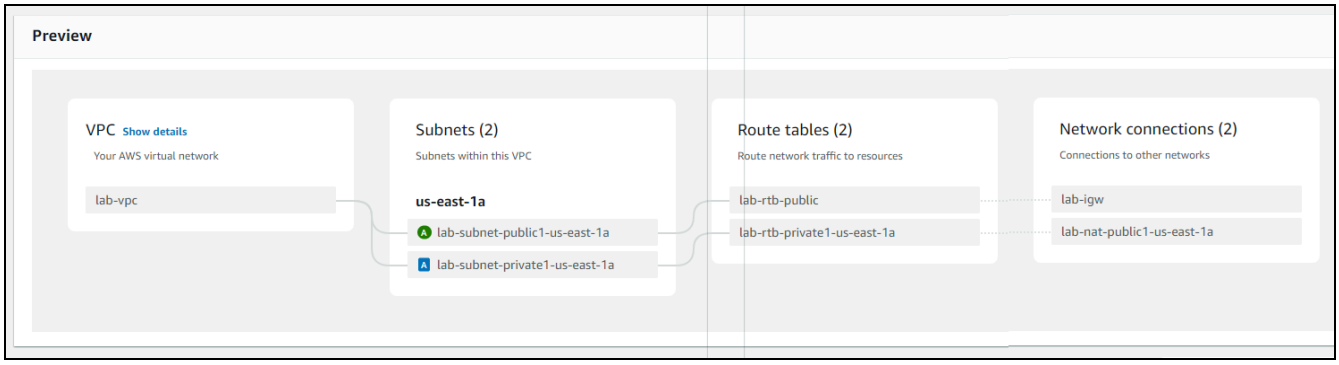


- Set **NAT gateways** to **In 1 AZ**.
- Set **VPC endpoints** to **None**.
- Keep both **DNS hostnames** and **DNS resolution** enabled.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

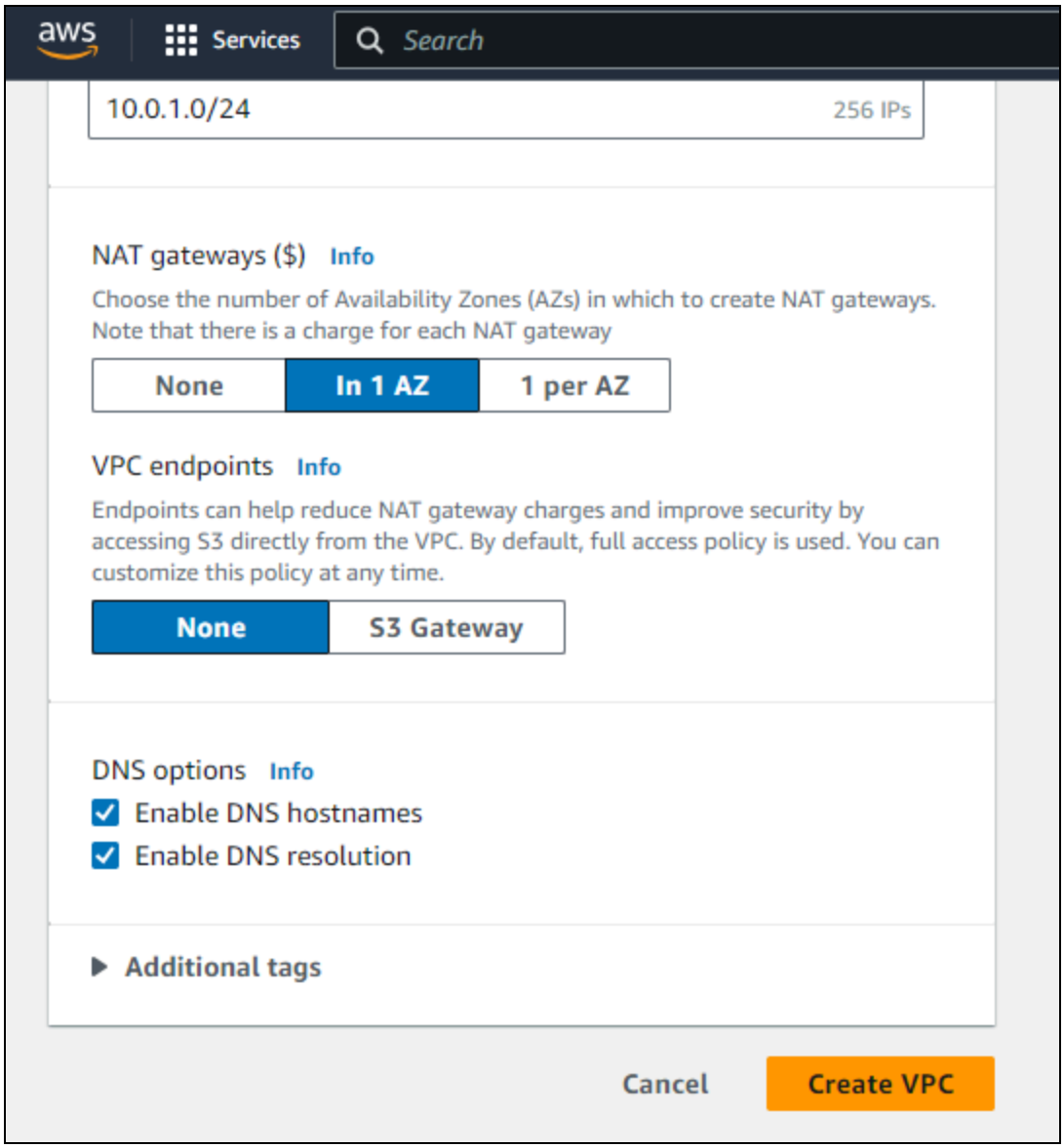


7. In the Preview panel on the right, confirm the settings you have configured.
    - **VPC**: `lab-vpc`
    - **Subnets**:
        - us-east-1a
            - Public subnet name: `lab-subnet-public1-us-east-1a`
            - Private subnet name: `lab-subnet-private1-us-east-1a`
    - **Route tables**
        - `lab-rtb-public`
        - `lab-rtb-private1-us-east-1a`
    - **Network connections**
        - `lab-igw`
        - `Lab-nat-public1-us-east-1a`



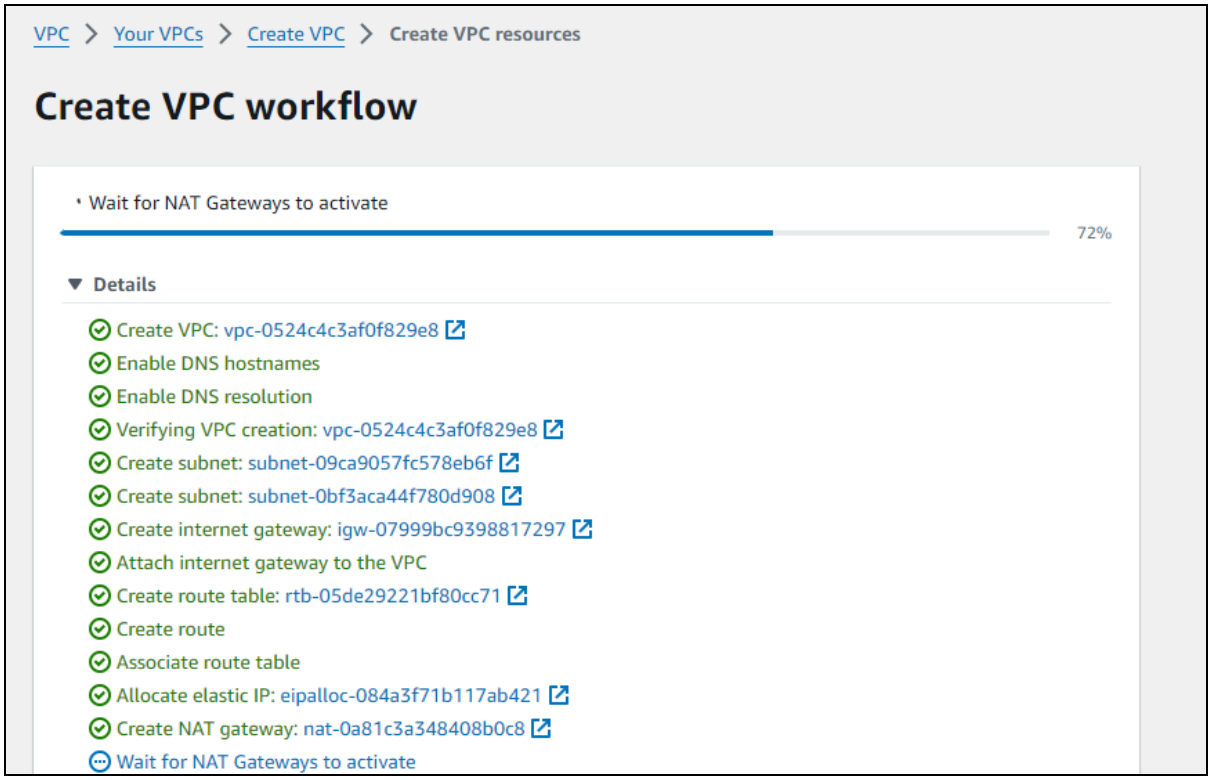
© Copyright upGrad Education Pvt. Ltd. All rights reserved

8. At the bottom of the screen, choose **Create VPC**

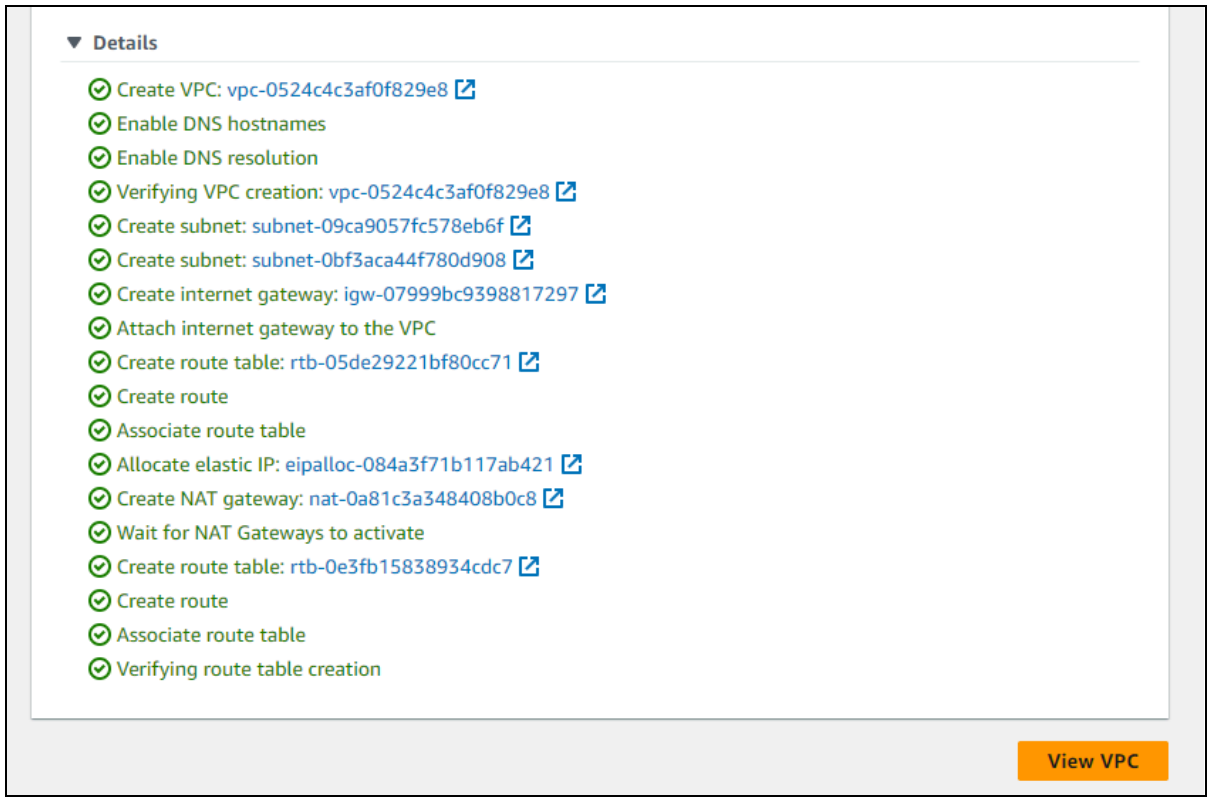


The VPC resources are created. The NAT Gateway will take a few minutes to activate. Please wait until all the resources are created before proceeding to the next step.

© Copyright upGrad Education Pvt. Ltd. All rights reserved



9. Once it is complete, choose **View VPC**



© Copyright upGrad Education Pvt. Ltd. All rights reserved

The wizard has provisioned a VPC with a public subnet and a private subnet in one Availability Zone with route tables for each subnet. It also created an Internet Gateway and a NAT Gateway.

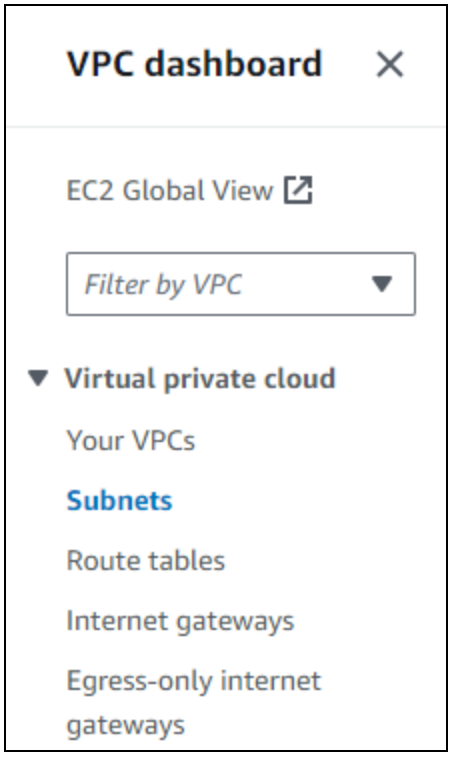
To view the settings of these resources, browse through the VPC console links that display the resource details. For example, choose **Subnets** to view the subnet details and choose **Route tables** to view the route table details.

Task 2: Create Additional Subnets

In this task, you will create two additional subnets for the VPC in a second Availability Zone. Having subnets in multiple Availability Zones within a VPC is useful for deploying solutions that provide High Availability.

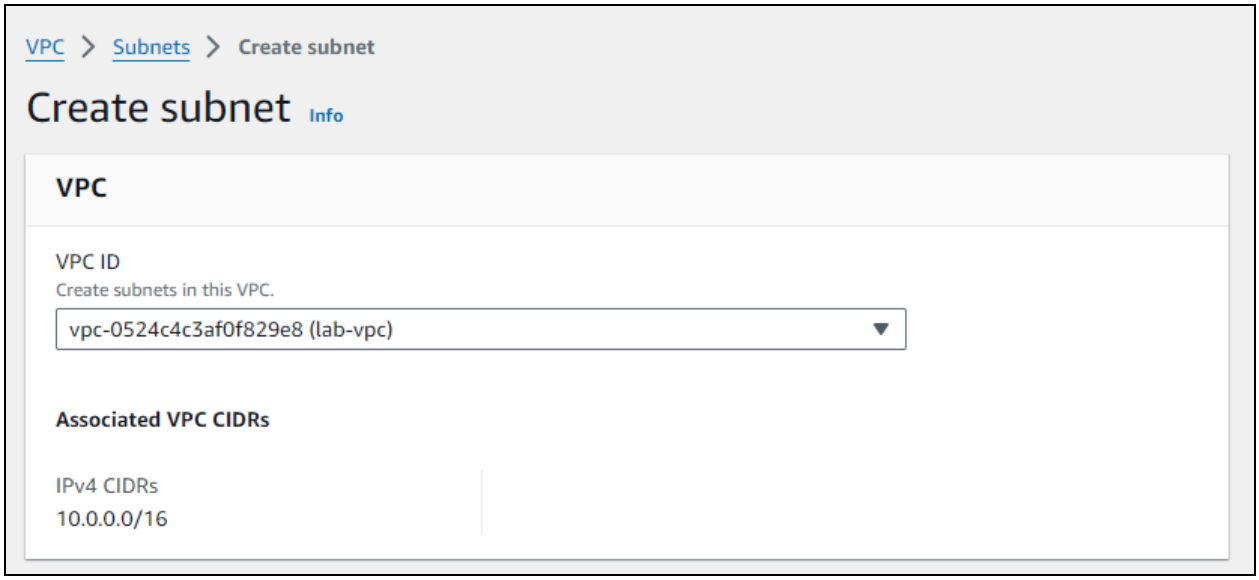
After creating a VPC as you have already done, you can still configure it further, for example, by adding more subnets. Each subnet you create resides entirely within one Availability Zone.

10. In the left navigation pane, choose **Subnets**.
    First, you will create a second public subnet.



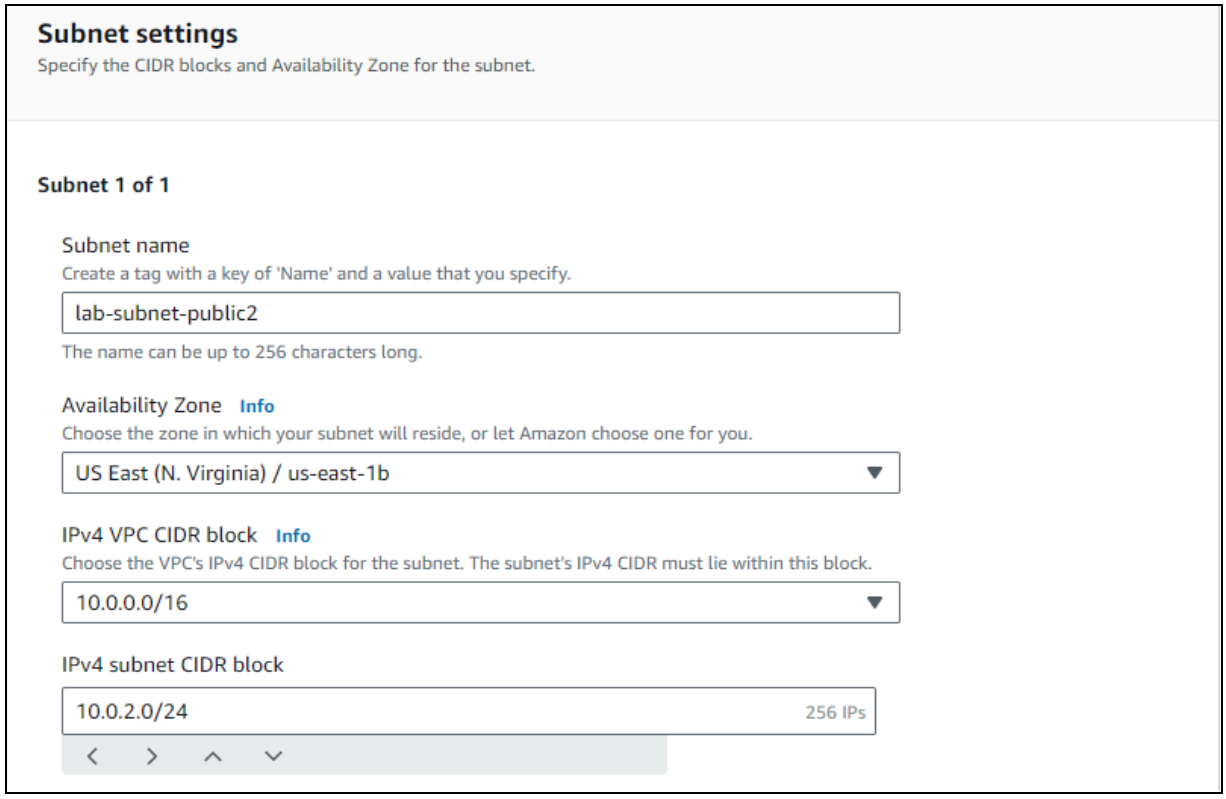
11. Choose **Create subnet** then configure:
    - **VPC ID: lab-vpc** (select from the menu).

© Copyright upGrad Education Pvt. Ltd. All rights reserved



- **Subnet name:** `lab-subnet-public2`
- **Availability Zone:** Select the second Availability Zone (for example, us-east-1b)
- **IPv4 VPC CIDR block:** `10.0.0.0/16` (select from the dropdown)
- **IPv4 subnet CIDR block:** `10.0.2.0/24`

The subnet will have all IP addresses starting with **10.0.2.x**.

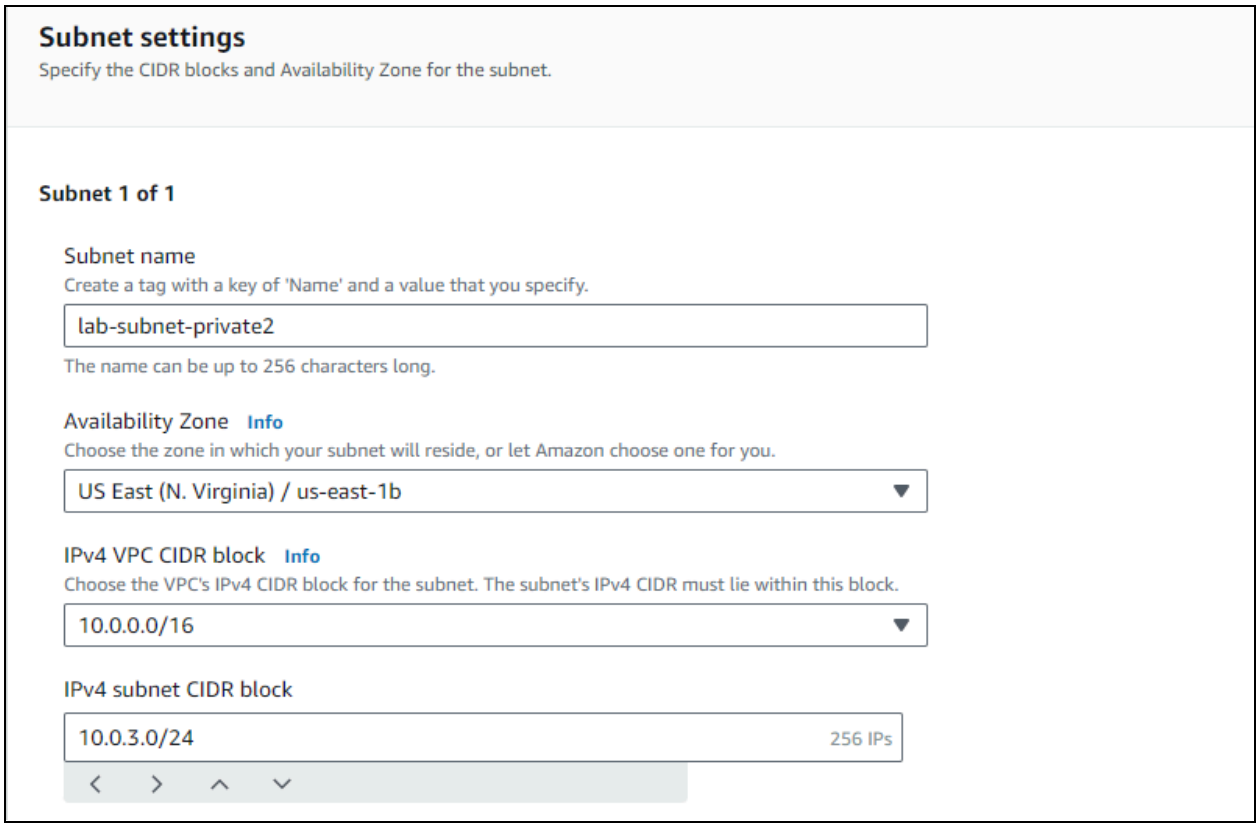


© Copyright upGrad Education Pvt. Ltd. All rights reserved

12. Choose **Create subnet**
    The second public subnet was created. You will now create a second private subnet.

13. Choose **Create subnet** then configure:
    - **VPC ID:** `lab-vpc`
    - **Subnet name:** `lab-subnet-private2`
    - **Availability Zone:** Select the *second* Availability Zone (for example, us-east-1b)
    - **IPv4 VPC CIDR block:** `10.0.0.0/16` (select from the dropdown)
    - **IPv4 CIDR block:** `10.0.3.0/24`

    The subnet will have all IP addresses starting with **10.0.3.x**.



14. Choose **Create subnet**

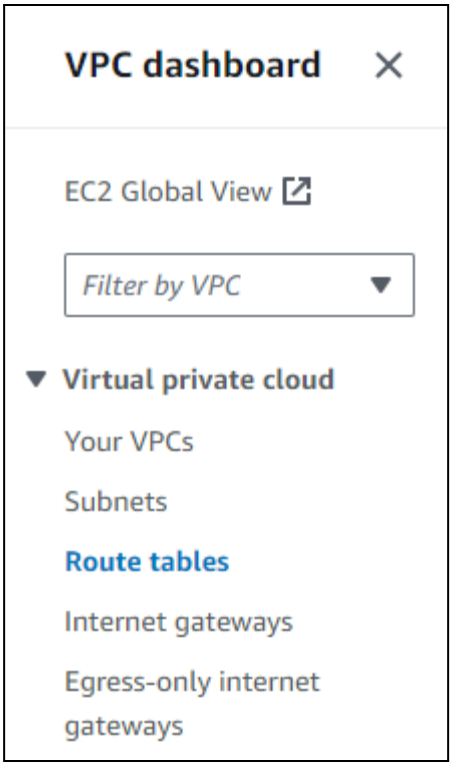
    The second private subnet was created.

    You will now configure this new private subnet to route internet-bound traffic to the NAT Gateway so that resources in the second private subnet are able to connect to the Internet, while still keeping the resources private. This is done by configuring a Route Table.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

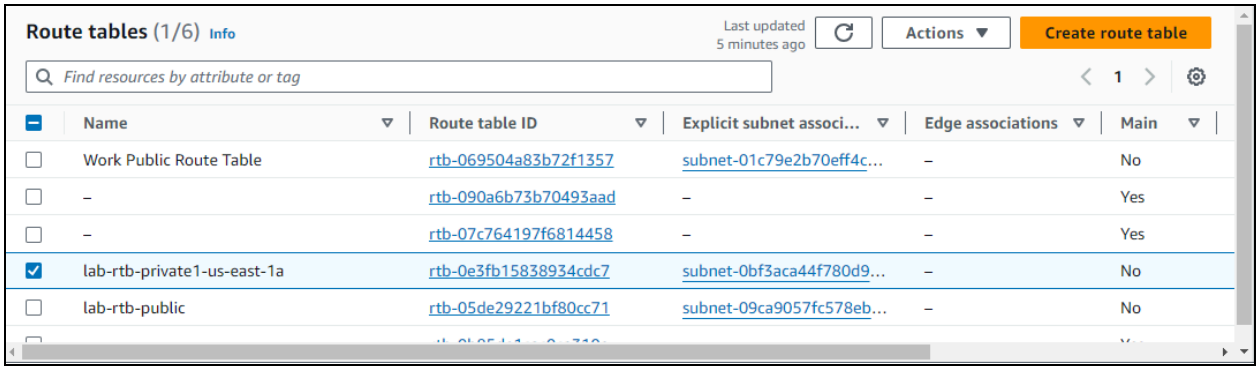
A *route table* contains a set of rules, called *routes*, that are used to determine where network traffic is directed. Each subnet in a VPC must be associated with a route table; the route table controls routing for the subnet.

15. In the left navigation pane, choose **Route tables**.



16. Select ☑ the **lab-rtb-private1-us-east-1a** route table.

    Note: If this is not present in the list, click on the □ refresh route tables button on the top left beside the **Create route table** button. If the issue still persists, wait for sometime and try refreshing again.

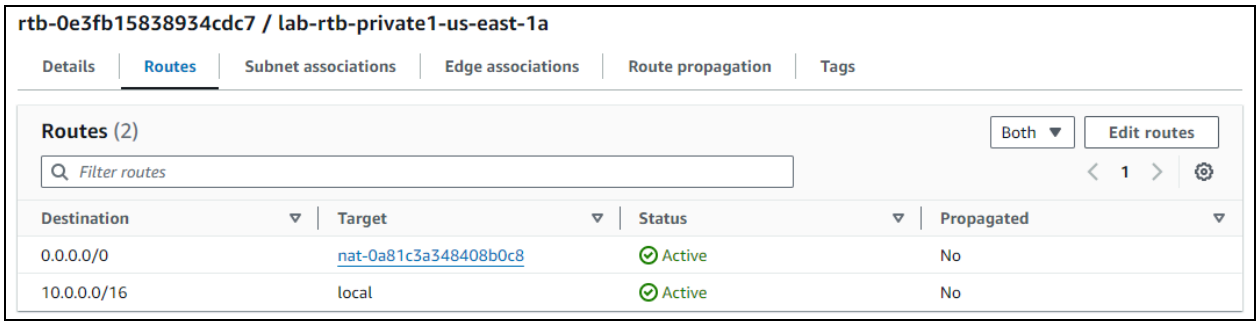


17. In the lower pane, choose the **Routes** tab.

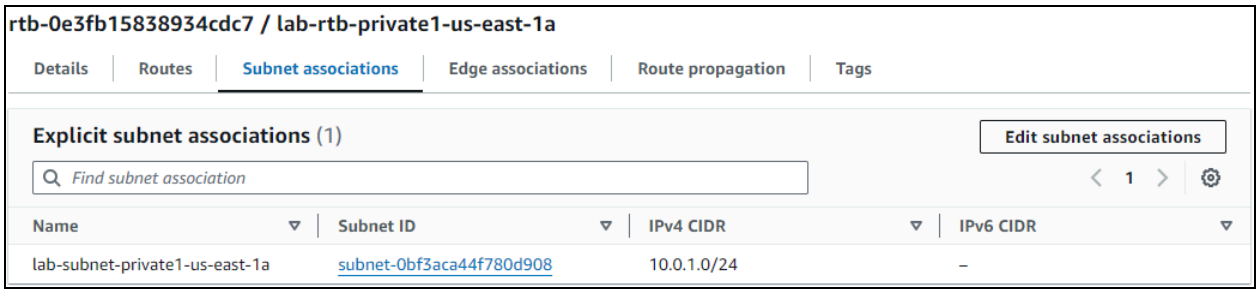
    Note that **Destination 0.0.0.0/0** is set to **Target nat-xxxxxxxx**. This means that traffic destined for the internet (0.0.0.0/0) will be sent to the NAT Gateway. The NAT Gateway

© Copyright upGrad Education Pvt. Ltd. All rights reserved

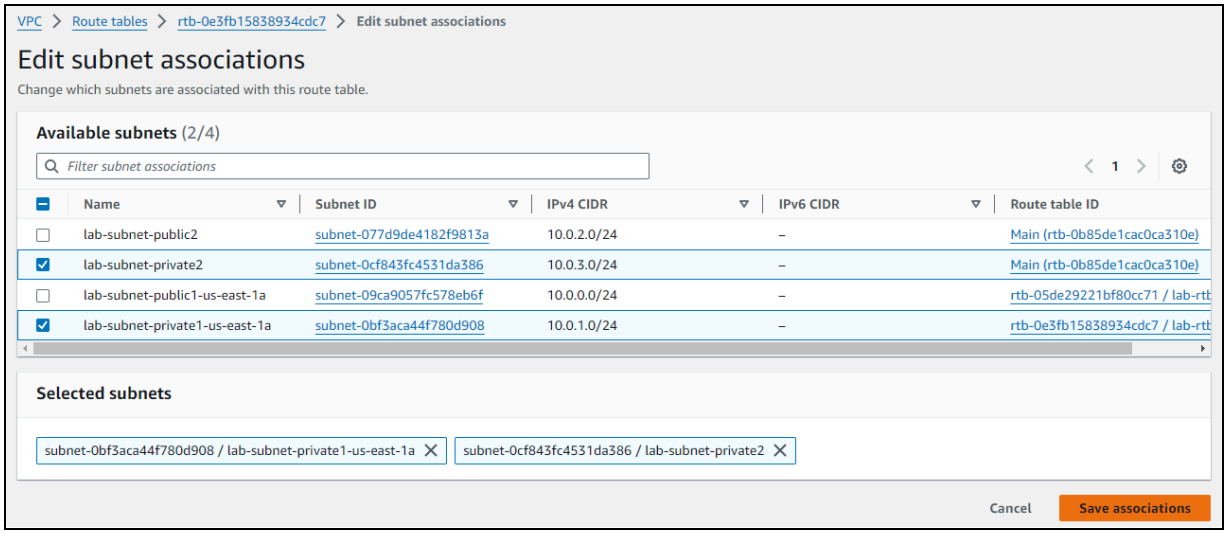
will then forward the traffic to the internet.
This route table is therefore being used to route traffic from private subnets.



18. Choose the **Subnet associations** tab.
    You created this route table in task 1 when you chose to create a VPC and multiple resources in the VPC. That action also created *lab-subnet-private-1* and associated that subnet with this route table.
    Now that you have created another private subnet, lab-subnet-private-2, you will associate this route table with that subnet as well.
19. In the Explicit subnet associations panel, choose **Edit subnet associations**



20. Leave **lab-subnet-private1-us-east-1a** selected, but also select ☑ **lab-subnet-private2**.
21. Choose **Save associations**

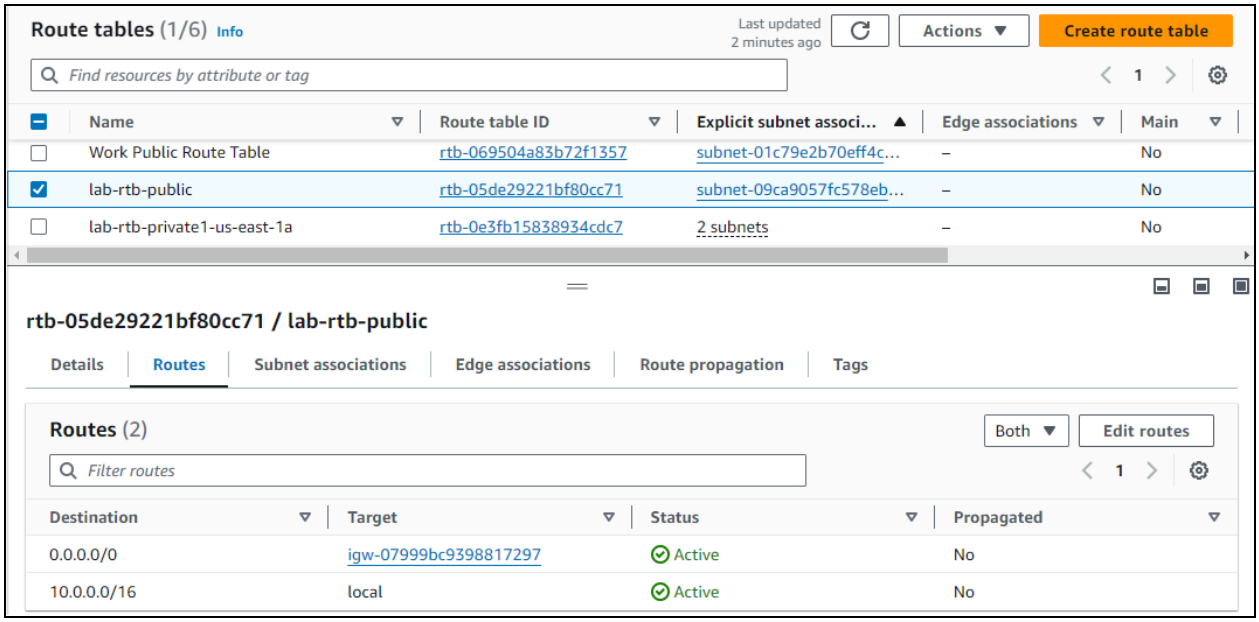


© Copyright upGrad Education Pvt. Ltd. All rights reserved

You will now configure the Route Table that is used by the Public Subnets.

22. Select the ☑ **lab-rtb-public** route table (and deselect any other subnets).

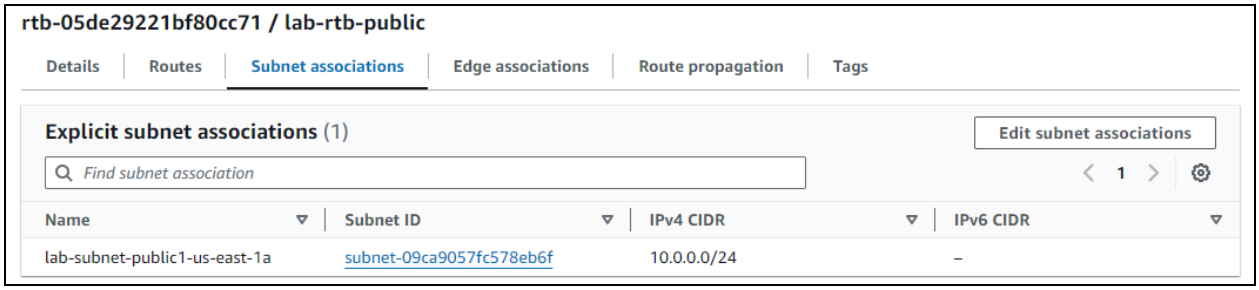
23. In the lower pane, choose the **Routes** tab.
    Note that **Destination 0.0.0.0/0** is set to Target **igw-xxxxxxxx**, which is an Internet Gateway. This means that internet-bound traffic will be sent straight to the internet via this Internet Gateway.



You will now associate this route table to the second public subnet you created.

24. Choose the **Subnet associations** tab.

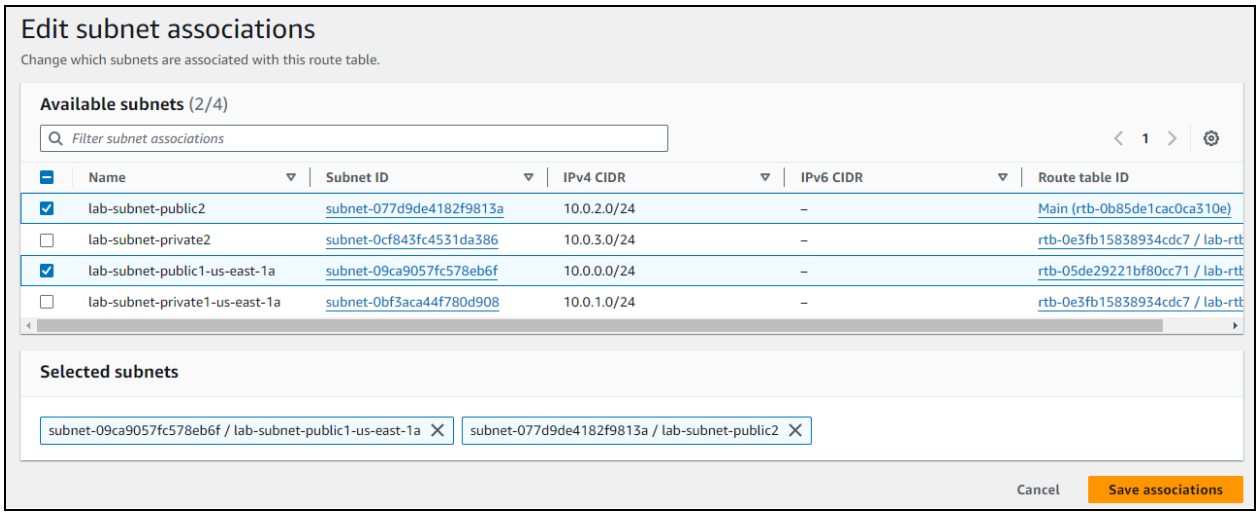
25. In the Explicit subnet associations area, choose **Edit subnet associations**



26. Leave **lab-subnet-public1-us-east-1a** selected, but also select ☑ **lab-subnet-public2**.

27. Choose **Save associations**

© Copyright upGrad Education Pvt. Ltd. All rights reserved

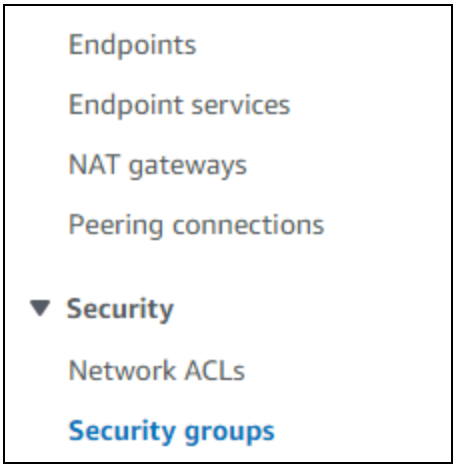


Your VPC now has public and private subnets configured in two Availability Zones. The route tables you created in task 1 have also been updated to route network traffic for the two new subnets.

Task 3: Create a VPC Security Group

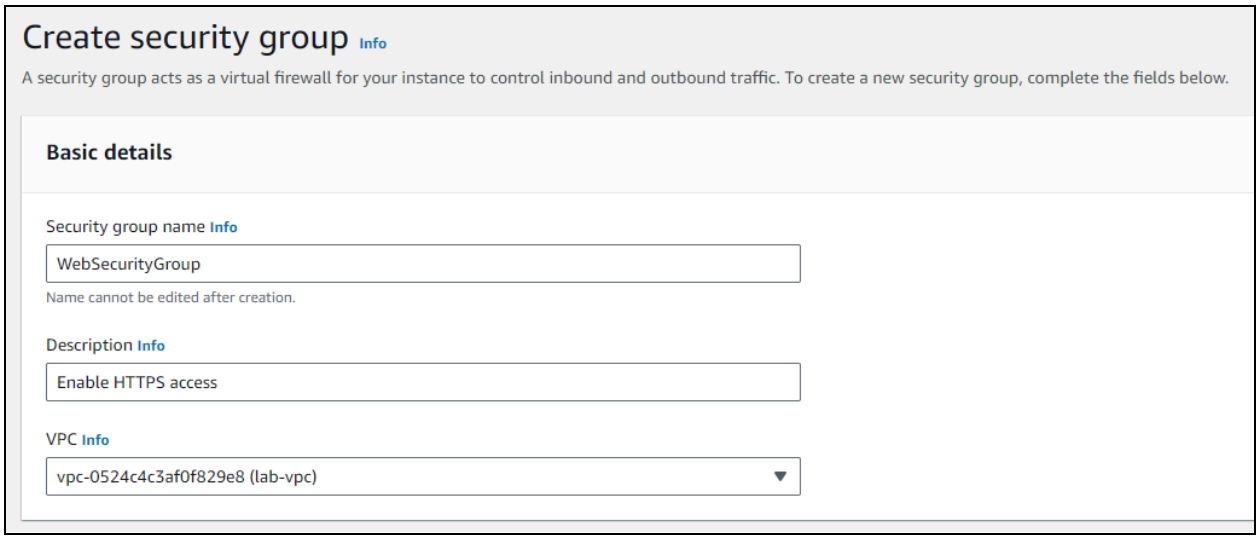
In this task, you will create a VPC security group, which acts as a virtual firewall. When you launch an instance, you associate one or more security groups with the instance. You can add rules to each security group that allow traffic to or from its associated instances.

28. In the left navigation pane, choose **Security groups**.



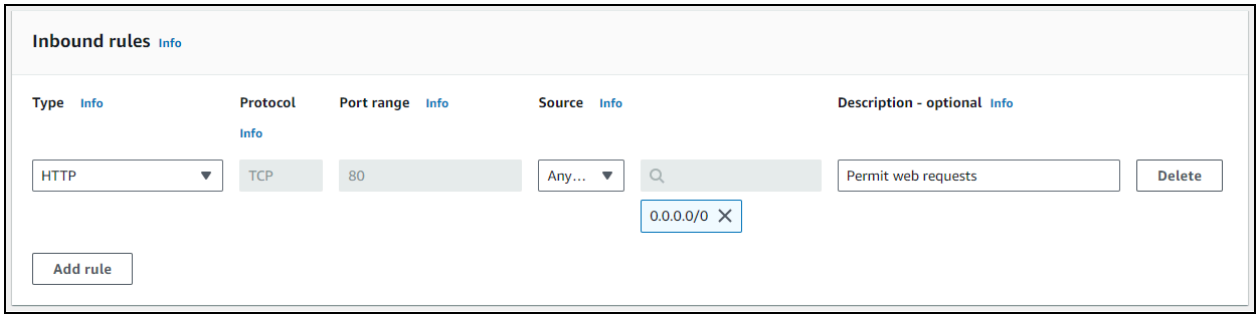
29. Choose **Create security group** and then configure:
    - Security group name: `WebSecurityGroup`
    - Description: `Enable HTTP access`
    - **VPC:** From the drop down list choose **lab-vpc**

© Copyright upGrad Education Pvt. Ltd. All rights reserved



30. In the Inbound rules pane, choose **Add rule**

31. Configure the following settings:
    - **Type:** *HTTP*
    - **Source:** *Anywhere-IPv4*
    - **Description:** `Permit web requests`



32. Scroll to the bottom of the page and choose **Create security group**

6. Introduction to Serverless Computing

You learned that serverless computing is a cloud computing execution model where the cloud provider automatically manages infrastructure as well as scaling and provisioning of servers. This model allows developers to focus solely on writing code and building applications without the need to manage or maintain servers. The primary characteristics of serverless computing include automatic scaling, built-in high availability and a pay-as-you-go billing model, where users are charged only for the compute time they consume.

The key concepts in serverless computing are as follows:

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- **Automatic scaling**: Serverless applications automatically scale in response to the number of requests or events. Manually provisioning or managing servers is not required, as the cloud provider handles the scaling based on demand.
- **Event-driven execution**: Serverless functions are typically triggered by events, such as changes in data, shifts in system state or user actions. These events can originate from various sources, including HTTP requests, database updates or file uploads.
- **Microservices architecture**: Serverless computing encourages the use of microservices, where applications are composed of small, independent functions that perform specific tasks. This modular approach simplifies development, testing and maintenance.
- **Cost efficiency**: With serverless computing, users only pay for the compute time used during the execution of their functions. There are no charges for idle time, making it a cost-effective solution for many applications.
- **Built-in high availability**: Serverless platforms offer high availability and fault tolerance out of the box. The cloud provider ensures that the infrastructure is highly available and resilient to failures, reducing the operational burden on developers.

6.1 Benefits

The benefits of serverless computing are as follows:

- **Reduced operational complexity**: By abstracting server management, developers can focus on writing and deploying code, leading to faster development cycles and reduced operational overhead.
- **Scalability**: Serverless applications automatically scale to handle varying workloads, ensuring optimal performance during peak times without manual intervention.
- **Cost savings**: The pay-as-you-go model ensures that users only pay for the actual compute time, reducing the costs associated with over-provisioning and idle resources.
- **Faster time to market**: With less time spent on infrastructure management, developers can accelerate the delivery of new features and applications.

Serverless computing represents a shift in how applications are built and deployed, offering a more agile, cost-effective and scalable approach to cloud computing. As a leading provider of serverless solutions, AWS offers services such as AWS Lambda, which empowers developers to harness the full potential of serverless computing.

6.2 Operational Model of Serverless Computing

You also learned about the operational model of serverless computing, which fundamentally changes how applications are developed, deployed and managed. Given below is an overview of the key components and operational workflow of the serverless computing model:

© Copyright upGrad Education Pvt. Ltd. All rights reserved

6.2.1 Event Sources

Serverless functions are triggered by events from various sources. These events can be from the following:

- **HTTP requests**: Triggered by API calls through services such as Amazon API Gateway
- **Database changes**: Triggered by changes in databases such as Amazon DynamoDB or Amazon RDS
- **File uploads**: Triggered by uploads to services such as Amazon S3
- **Scheduled events**: Triggered by time-based events using services such as Amazon CloudWatch Events
- **Messaging services**: Triggered by messages from Amazon SNS or Amazon SQS

6.2.2 Serverless Functions

The core component of serverless computing is the serverless function. These functions are small, single-purpose code units that execute in response to events. For example, AWS Lambda functions are designed for the following:

- To process data streams
- To handle HTTP requests
- To respond to database updates
- To perform file transformations

6.2.3 Execution Environment

When an event triggers a serverless function, the cloud provider automatically provisions the necessary compute resources, runs the function and deallocates the resources once the execution is complete. This environment includes the following:

- **Automatic provisioning**: The cloud provider automatically provisions and scales the necessary compute resources based on the workload.
- **Ephemeral containers**: Functions run in isolated, short-lived containers that are created on demand and terminated once the function completes.

6.2.4 Stateless Architecture

Serverless functions are inherently stateless, meaning each invocation is independent and does not retain any state from previous invocations. To manage the state, serverless applications often rely on external storage and databases.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

6.2.5 Security and Permissions

Security in serverless computing is managed through fine-grained permissions and roles, which are given below:

- **IAM roles and policies**: Define which resources a function can access and what actions it can perform.
- **Built-in security features**: Functions run in isolated environments, reducing the risk of security breaches.

6.2.6 Deployment and Management

Serverless applications are deployed and managed using infrastructure as code (IaC) tools and continuous integration/continuous deployment (CI/CD) pipelines:

- **AWS CloudFormation and AWS SAM**: Used for defining and deploying serverless applications.
- **CI/CD pipelines**: Automate the deployment process, ensuring rapid and reliable updates.

7. AWS Lambda Basics

You learned that AWS Lambda is a serverless compute service that enables developers to run code without provisioning or managing servers. Lambda executes your code only when triggered by events and automatically scales to handle varying levels of demand. This service simplifies the process of building scalable and fault-tolerant applications, as it abstracts away the underlying infrastructure management.

Lambda functions are event-driven, meaning they run in response to events from other AWS services or direct invocations using AWS SDKs. These events can originate from sources such as changes in data within Amazon S3 buckets, updates to Amazon DynamoDB tables, incoming messages from Amazon SNS or Amazon SQS and HTTP requests via Amazon API Gateway. Each Lambda function is executed within a secure, isolated runtime environment that includes the necessary resources and dependencies specified by the developer.

7.1 Features

- **Event-driven execution**: Lambda functions are triggered by various events from AWS services or direct invocations, allowing for highly responsive applications.
- **Automatic scaling**: Functions scale automatically with the number of incoming events, providing seamless scalability without manual intervention.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- **Granular billing**: Pay only for the compute time used, calculated in milliseconds and no charges are incurred when your code is not running.
- **Integrated security**: Secure execution environments managed by AWS, with permissions controlled via AWS Identity and Access Management (IAM).

7.2 Supported Programming Languages

AWS Lambda supports a variety of programming languages, giving developers the flexibility to write functions in their preferred language. Supported languages include the following:

- Python
- Java
- Node.js
- C#
- Ruby
- Go
- PowerShell
- Custom runtimes: Developers can implement their own runtime if it is not natively supported by AWS Lambda.

7.3 Use Cases

- **Data processing**
  AWS Lambda can process data in real-time from various sources. For example, Lambda can be triggered to process log files uploaded to an Amazon S3 bucket or to handle real-time data streams from Amazon Kinesis.
- **Web applications**
  Build scalable web backends and APIs using Lambda in conjunction with Amazon API Gateway. Lambda can handle HTTP requests, process user inputs and connect to backend databases, providing a robust backend for web applications.
- **Automation**
  Automate routine tasks such as resizing images uploaded to Amazon S3, managing AWS resources or performing scheduled tasks using Amazon CloudWatch Events to trigger Lambda functions at specified times.
- **Machine Learning**
  Lambda can be used to run inference on machine learning models. For instance, Lambda can be triggered to process and analyse data in real-time for anomaly detection or predictive analytics.

AWS Lambda provides a powerful, scalable and cost-effective platform for building a wide range of applications without the need to manage underlying infrastructure. By leveraging Lambda,

© Copyright upGrad Education Pvt. Ltd. All rights reserved

developers can focus on writing code and developing features while AWS handles the operational aspects, scaling and maintenance.

8. Integrating AWS Lambda with Other AWS Services

You learned that AWS Lambda seamlessly integrates with a variety of AWS services, enabling developers to build complex, event-driven applications that respond to changes in their AWS environment. Here are some of the most common integrations:

1. **Amazon S3**
   Lambda can be triggered by events in an Amazon S3 bucket, such as when an object is created, deleted or modified. This integration is useful for processing uploaded files, generating thumbnails and other data transformation tasks.
2. **Amazon DynamoDB**
   Lambda can respond to changes in Amazon DynamoDB tables. For example, a Lambda function can be triggered whenever an item is added, updated or deleted in a DynamoDB table, allowing for real-time data processing, synchronisation and analytics.
3. **Amazon API Gateway**
   API Gateway can be used to create RESTful APIs that trigger Lambda functions. This integration allows developers to build scalable, serverless backends for web and mobile applications that process incoming HTTP requests, perform logic and return responses.
4. **Amazon Simple Notification Service (SNS)**
   Lambda can be triggered by messages published to an Amazon SNS topic. This integration is useful for implementing event-driven architectures, where Lambda functions handle notifications and alerts, process messages or initiate other workflows.
5. **AWS CloudWatch**
   AWS CloudWatch can trigger Lambda functions based on scheduled events (using CloudWatch Events) or in response to specific metrics and alarms. This allows for automated responses to changes in system performance, periodic tasks and real-time monitoring.

8.1 Use Cases

1. **Real-Time File Processing With Amazon S3**
   - To automatically process and analyse data as it is uploaded to an S3 bucket
   - **Example**: A Lambda function is triggered whenever a new log file is uploaded to an S3 bucket. The function processes the log data, extracts meaningful information and stores the results in an Amazon DynamoDB table for further analysis
2. **Database Trigger With Amazon DynamoDB**

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- To synchronise data between systems or perform real-time analytics
- **Example**: A Lambda function is triggered by a DynamoDB stream whenever new sales data is added to a table. The function processes the sales data, updates inventory levels and sends a summary to an Amazon SNS topic to notify the sales team

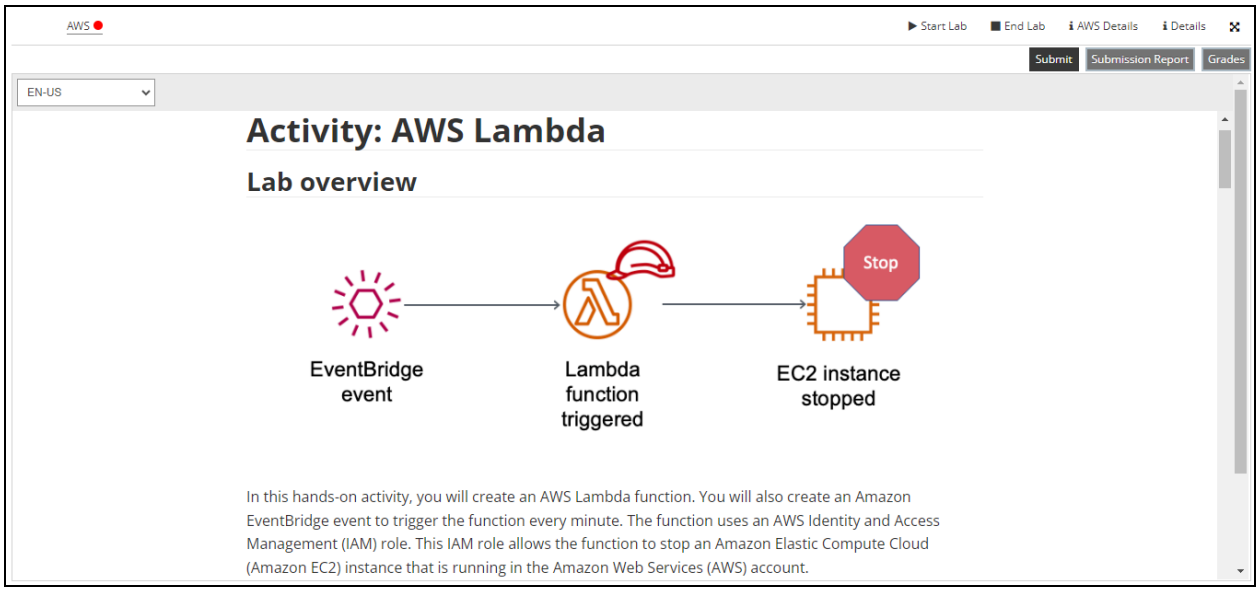
3. **Serverless Web Application Backend With API Gateway**
   - To handle HTTP requests for a web application without managing servers
   - **Example**: An e-commerce website uses API Gateway to expose RESTful endpoints. Lambda functions are triggered by these endpoints to handle user authentication, process orders and interact with a backend database
4. **Event-Driven Notifications With Amazon SN**
   - To send notifications and alerts based on events in the system.
   - **Example**: An IoT application publishes sensor data to an SNS topic. A Lambda function is triggered by this topic to analyse the data and send an alert if certain thresholds are exceeded, ensuring timely notifications of critical events
5. **Automated Monitoring and Response With AWS CloudWatch**
   - To automate responses to system performance changes and scheduled tasks
   - **Example**: A Lambda function is scheduled to run every hour using CloudWatch Events. The function checks the health of various system components, logs the results and triggers alarms if any issues are detected. Additionally, CloudWatch metrics can trigger Lambda functions to auto-scale resources or perform other corrective actions in real-time

By integrating AWS Lambda with these and other AWS services, developers can build highly responsive, scalable and cost-effective applications. These integrations enable Lambda to act as the glue between different services, facilitating seamless data flow and automation across the AWS ecosystem.

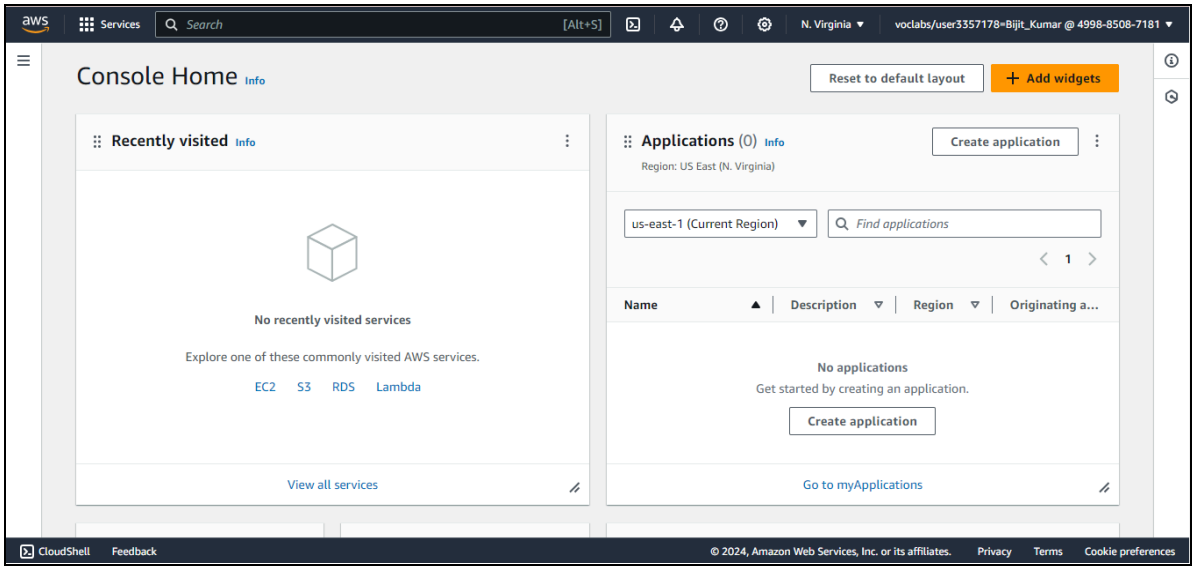
9. Hands-on Lab/Demo: Creating and Deploying Your First Lambda Function

1. At the top of these instructions, choose **Start Lab**.
   - The lab session starts.
   - A timer displays at the top of the page and shows the time remaining in the session.
     Tip: To refresh the session length at any time, choose **Start Lab** again before the timer reaches 0:00.
   - Before you continue, wait until the circle icon to the right of the AWS 🟢 link in the upper-left corner turns green.

© Copyright upGrad Education Pvt. Ltd. All rights reserved



2. To access the AWS Management Console, click on the **AWS** link located in the top left corner. This will open a new tab in your browser, directing you to the console.
Tip: If a new tab does not appear, your browser may be blocking pop-ups. Typically, a message or icon indicating this will appear at the top of your browser. Click on this message or icon, and select **Allow pop-ups** to proceed.

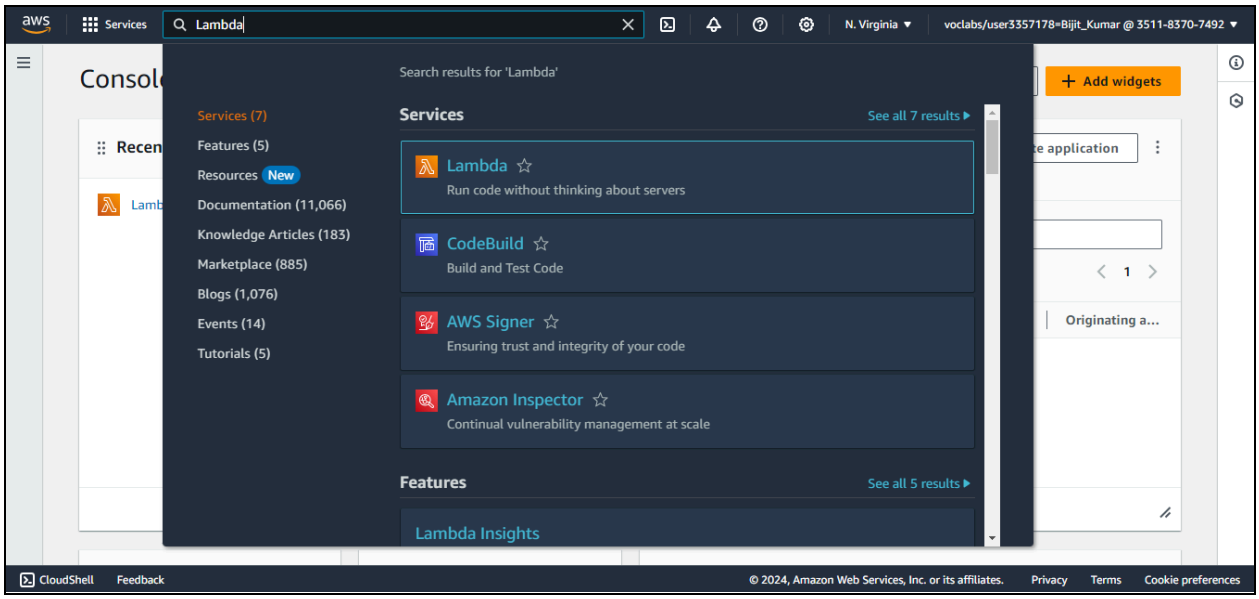


3. For convenience, arrange the AWS Management Console tab next to this set of instructions. By viewing both tabs simultaneously, you can easily follow the steps of the lab.

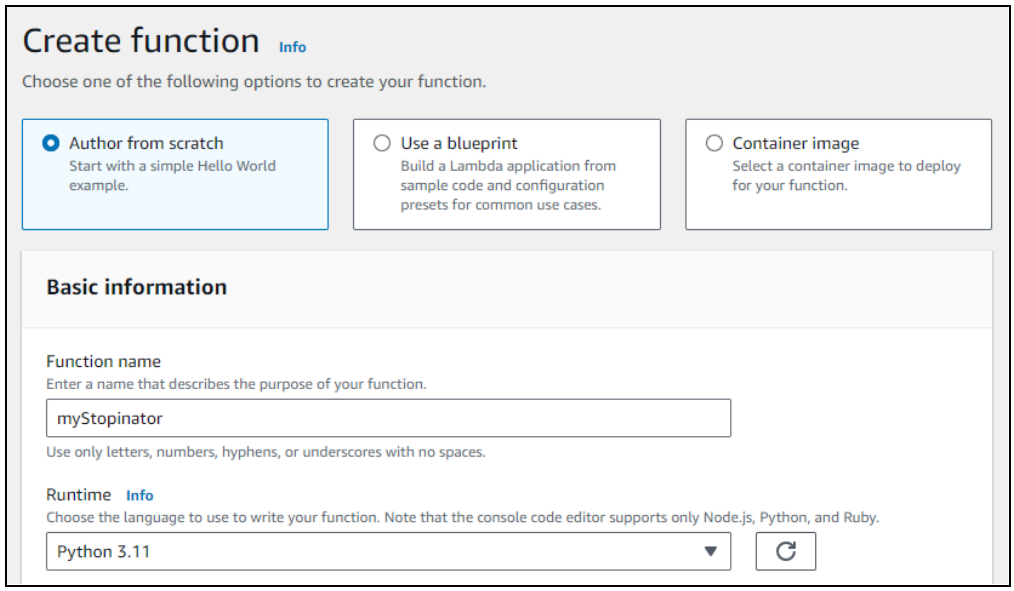
© Copyright upGrad Education Pvt. Ltd. All rights reserved

Task 1: Create a Lambda function

4. Locate the search box near the **Services** label and enter **Lambda** into it. Select **Lambda** from the search results to open the AWS Lambda console.

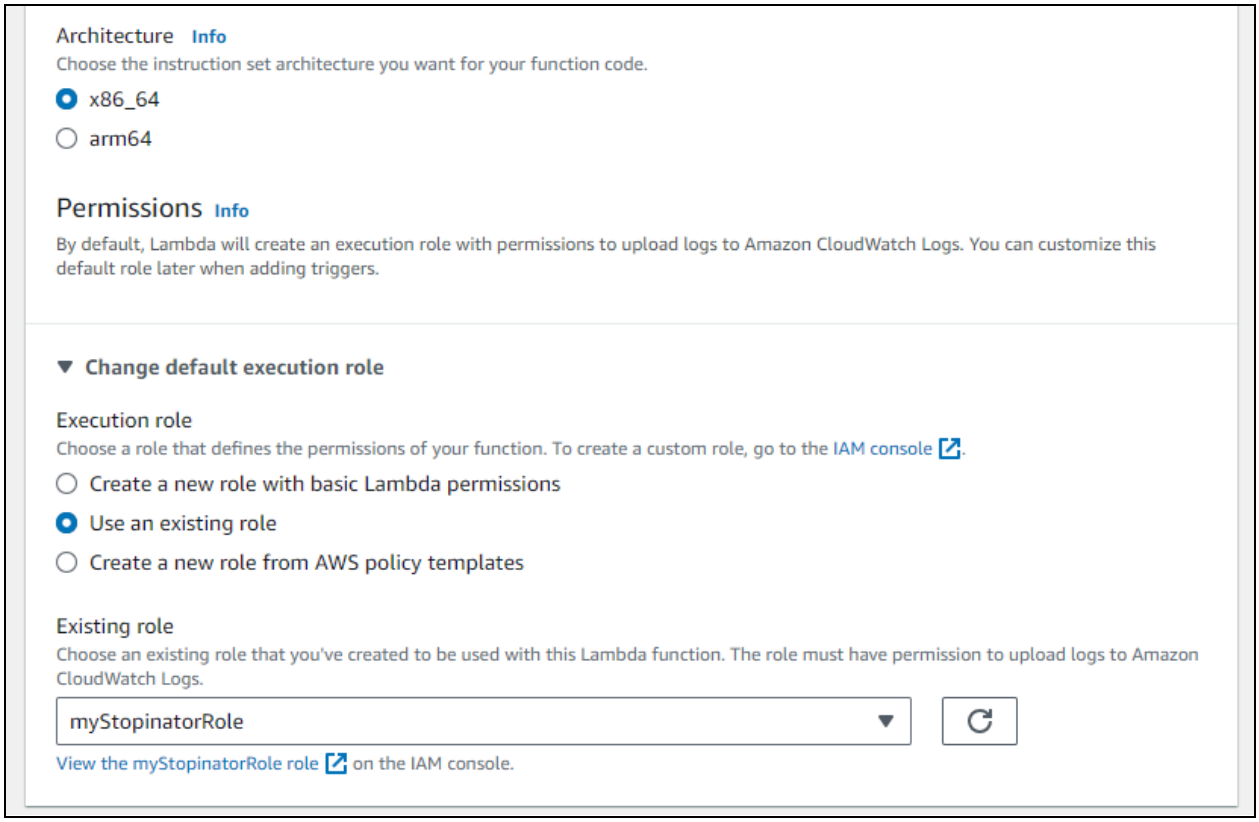


5. Choose **Create a function**.
6. In the **Create function** screen, configure these settings:
    - Choose **Author from scratch**
    - Function name: `myStopinator`
    - Runtime: **Python 3.11**



© Copyright upGrad Education Pvt. Ltd. All rights reserved

- Architecture: **x86_64**
- Choose **Change default execution role**
  - Execution role: **Use an existing role**
  - Existing role: From the dropdown list, choose **myStopinatorRole**



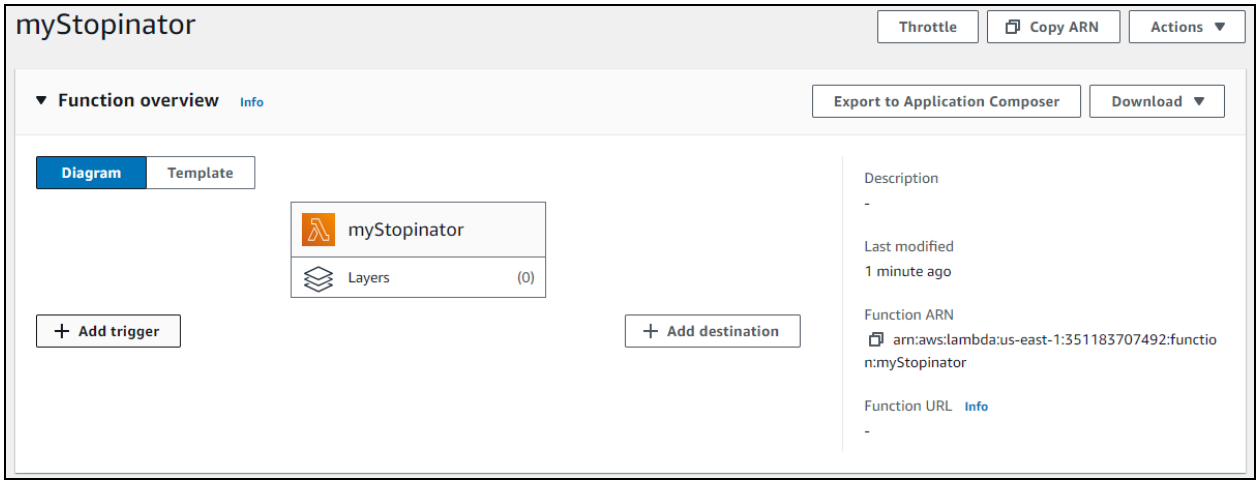
7. Choose **Create function**.

Task 2: Configure the trigger

In this task, you will configure a scheduled event to trigger the Lambda function by setting an Amazon EventBridge event as the event source (or trigger). The Lambda function can be configured to operate much like a cron job on a Linux server or a scheduled task on a Microsoft Windows server. However, you do not need to have a server running to host it.

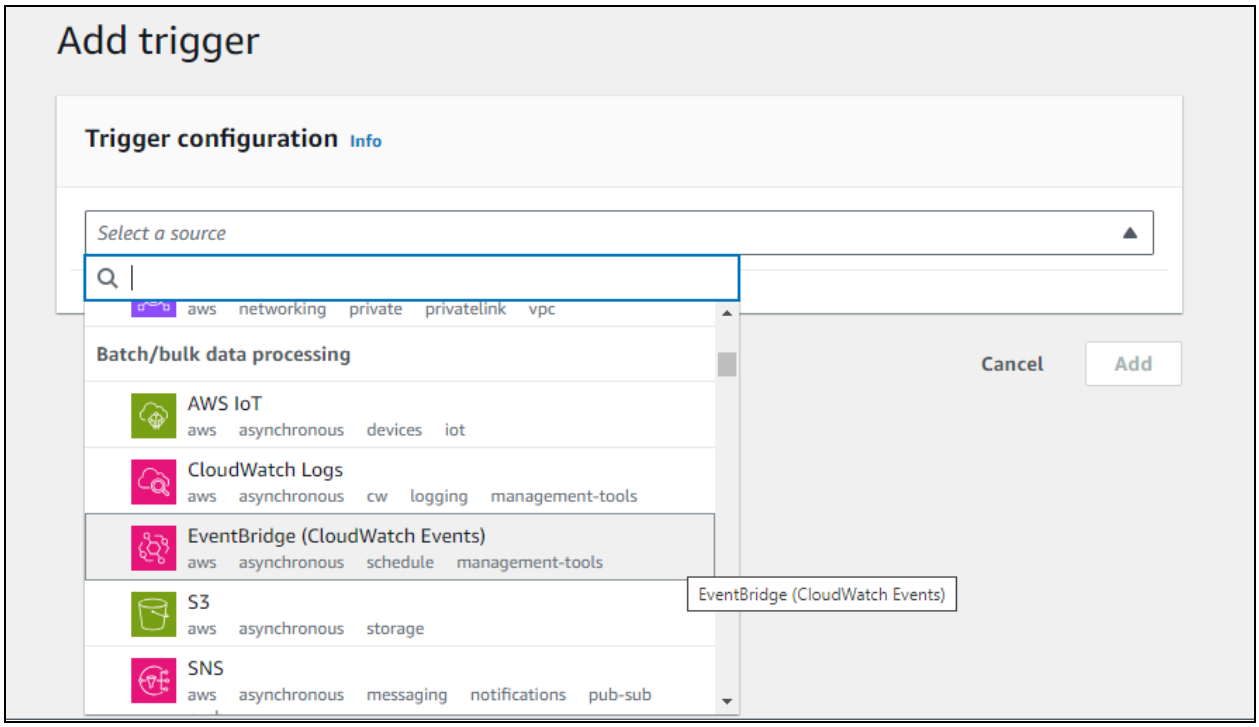
8. Choose **Add trigger**.

© Copyright upGrad Education Pvt. Ltd. All rights reserved



9. In the **Trigger configuration**, choose the **Select a source** dropdown menu, and select **EventBridge (CloudWatch Events)**.

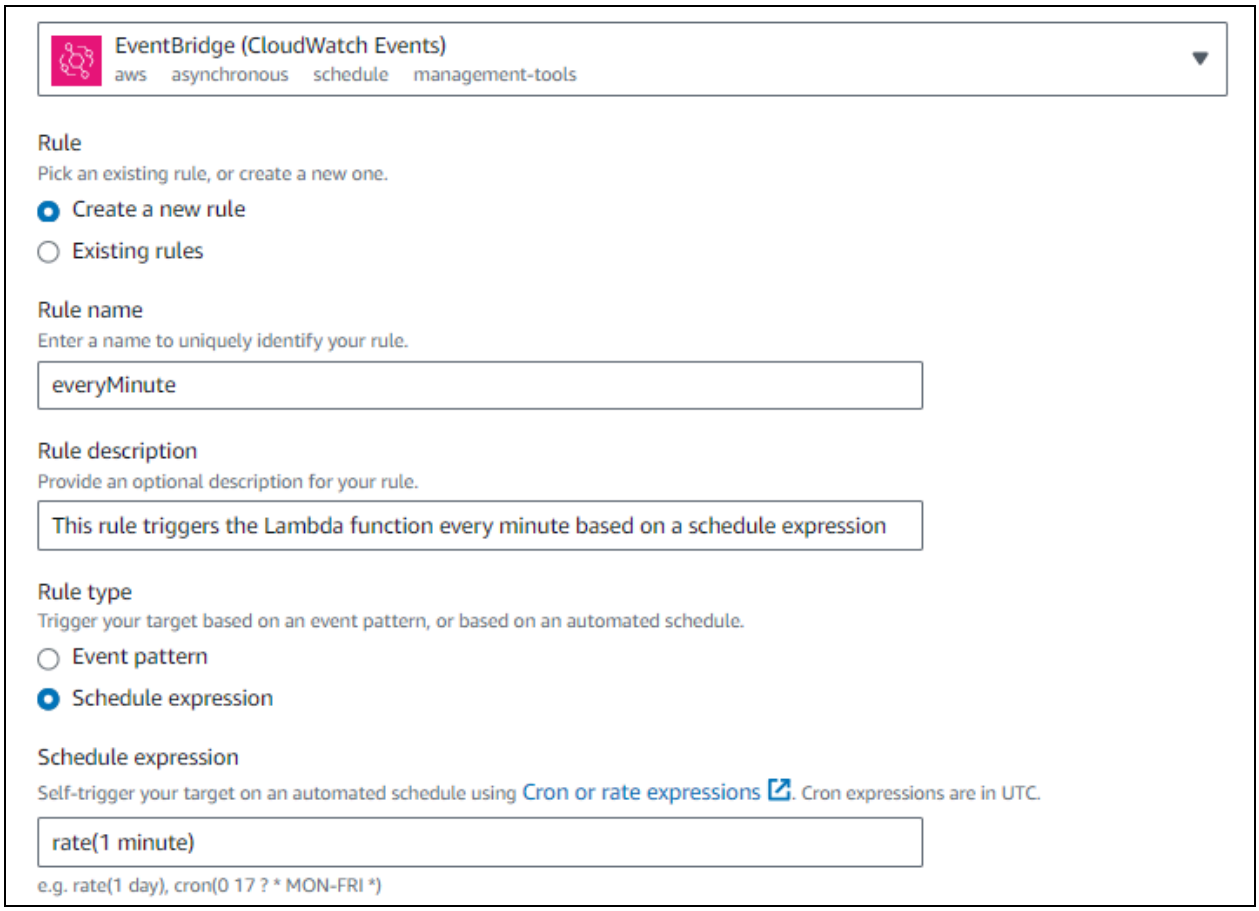
   Note: If you are unable to find **EventBridge (CloudWatch Events)**, search for it instead in the search box in the dropdown menu.



10. For the rule, select **Create a new rule** and configure these settings:
    - Rule name: `everyMinute`
    - Rule description: `This rule triggers the Lambda function every minute based on a schedule expression.`
    - Rule type: **Schedule expression**

© Copyright upGrad Education Pvt. Ltd. All rights reserved

- Schedule expression: `rate(1 minute)`



Note: A more realistic, schedule-based stopinator Lambda function would probably be triggered by using a cron expression instead of a rate expression. However, for the purposes of this activity, using a rate expression ensures that the Lambda function will be triggered soon enough that you can see the results.

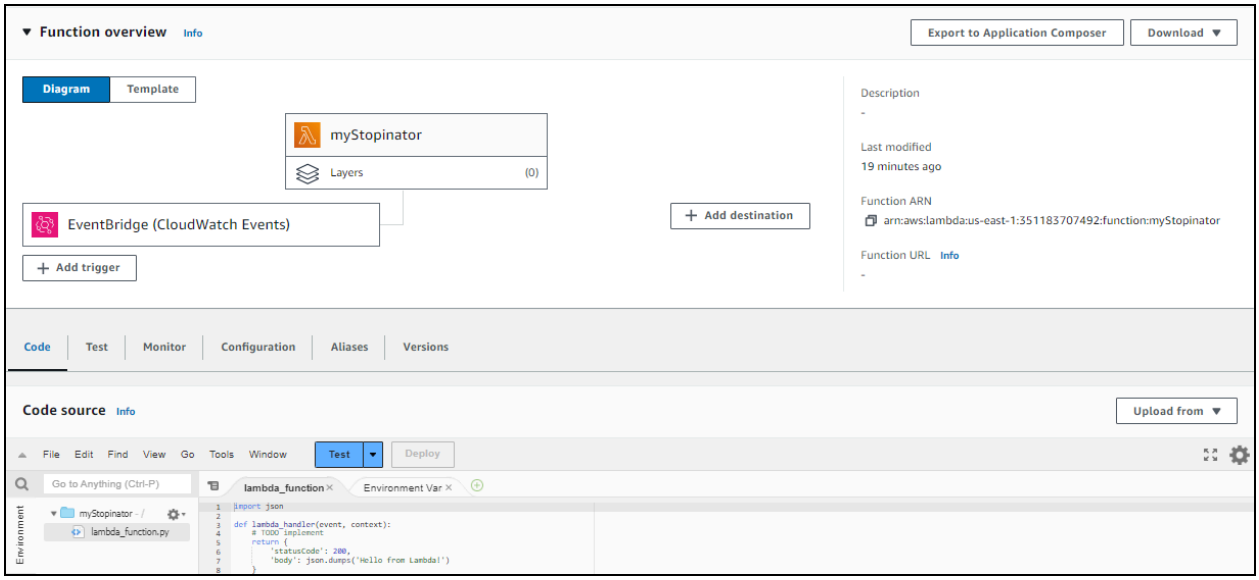
11. Choose **Add**.

Task 3: Configure the Lambda function

In this task, you will paste a few lines of code to update two values in the function code. You do not need to write code to complete this task.

12. Below the **Function overview** pane, choose **Code**, and then choose *lambda_function.py* to display and edit the Lambda function code.

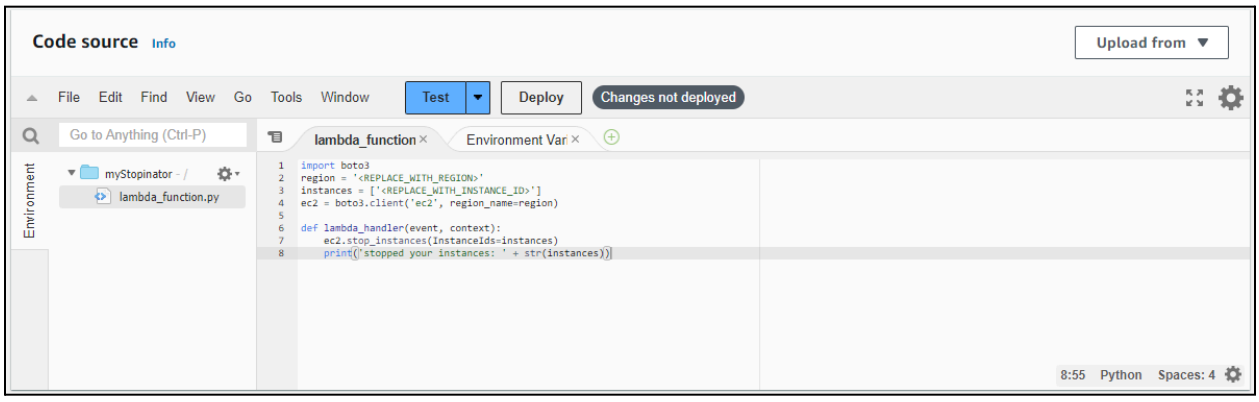
© Copyright upGrad Education Pvt. Ltd. All rights reserved



13. In the **Code source** pane, delete the existing code. Copy the following code, and paste it in the box:

```
import boto3
region = '<REPLACE_WITH_REGION>'
instances = ['<REPLACE_WITH_INSTANCE_ID>']
ec2 = boto3.client('ec2', region_name=region)

def lambda_handler(event, context):
   ec2.stop_instances(InstanceIds=instances)
   print('stopped your instances: ' + str(instances))
```

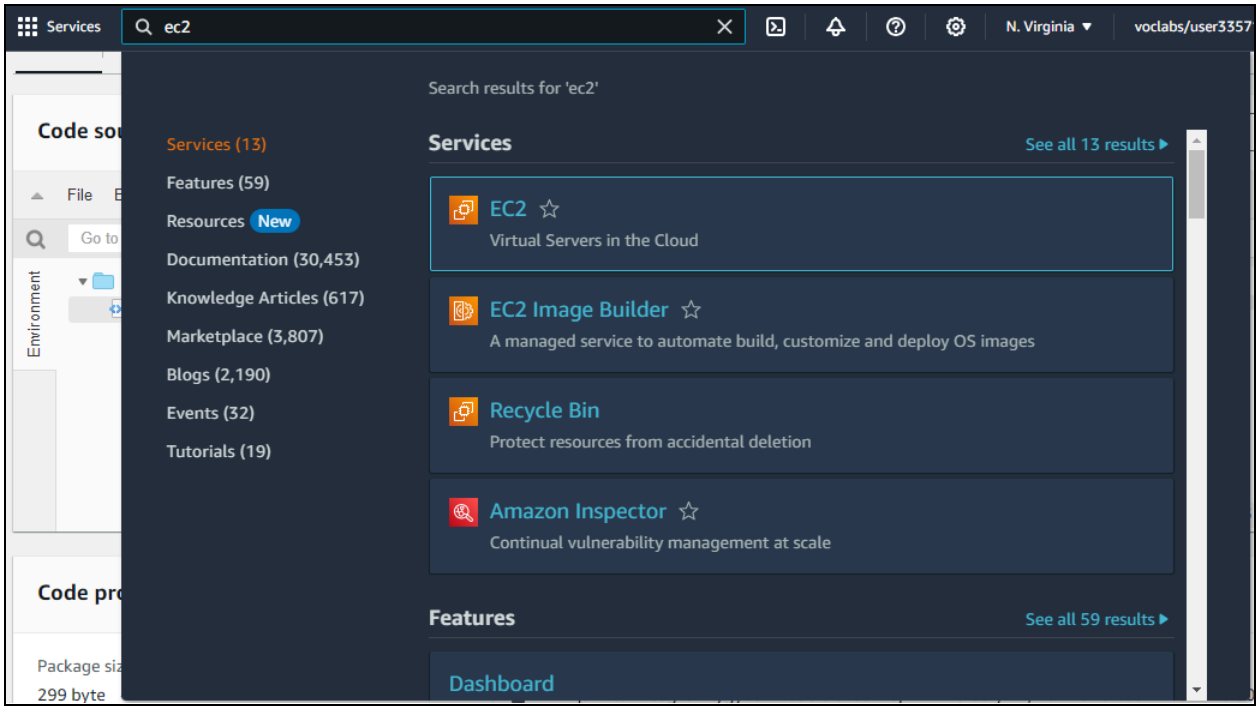


14. Replace the `<REPLACE_WITH_REGION>` placeholder with the actual Region that you are using. To do this:
Choose the region on the top right corner and use the region code. For example, the

© Copyright upGrad Education Pvt. Ltd. All rights reserved

region code for US East (N. Virginia) is *us-east-1*.
Important: Keep the single quotation marks (' ') around the Region in your code. For example, for the N. Virginia, it would be `'us-east-1'`

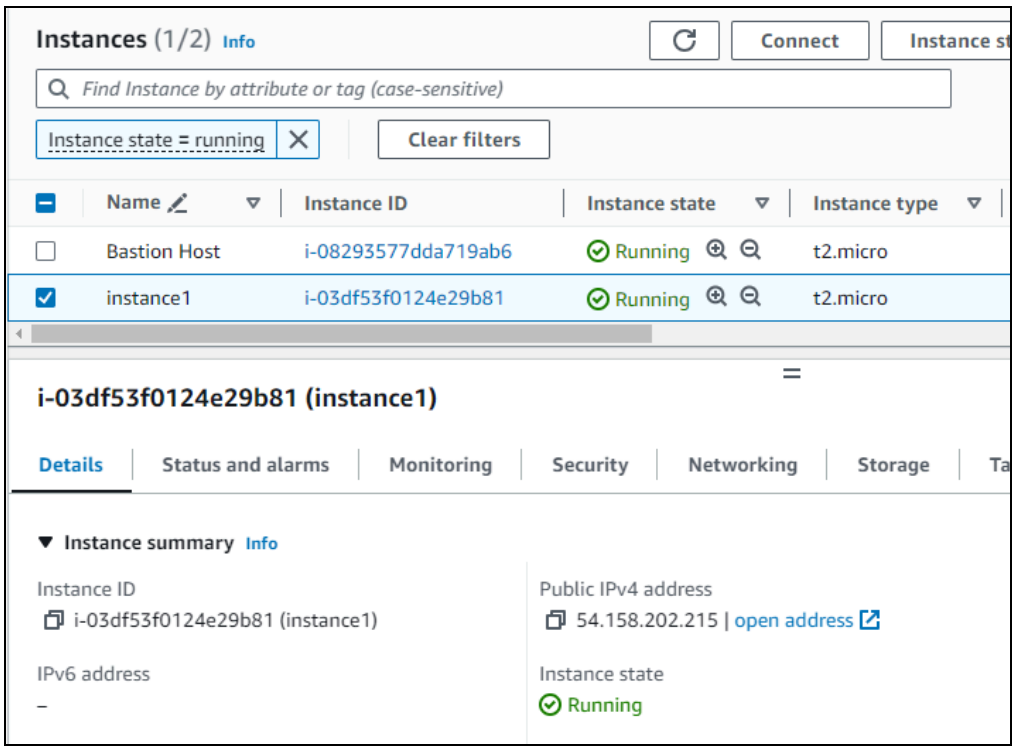
15. In the search box near the **Services** label, enter **ec2** and open the **EC2** from the search results in another tab OR go to https://console.aws.amazon.com/ec2.



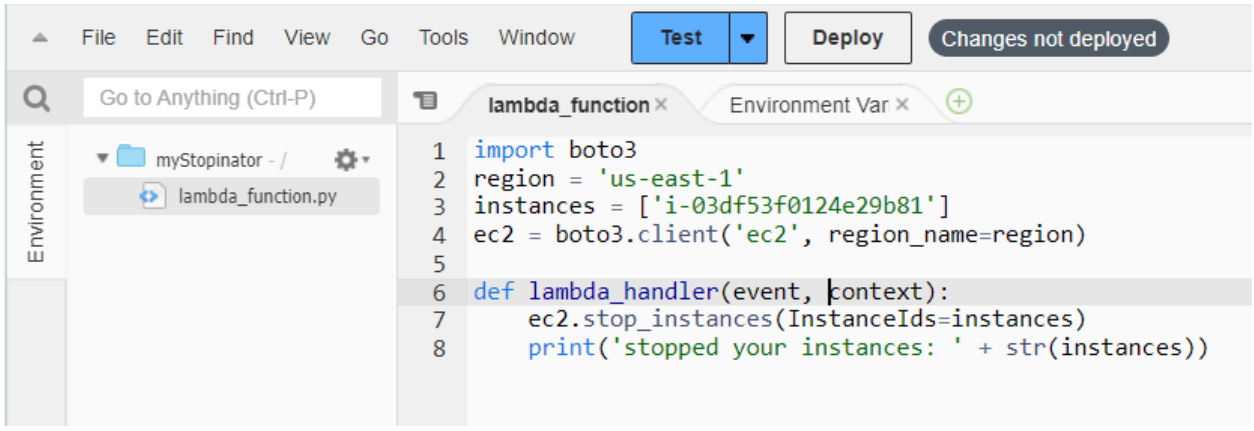
16. Choose **Instances**. Note that an EC2 instance named **instance1** exists and that it is in a running state. Select the box beside the instance1 and from the Details tab of instance1, copy the **Instance ID** (it will start with i-)

    Note: Leave this browser tab open. You will return to it in a moment.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

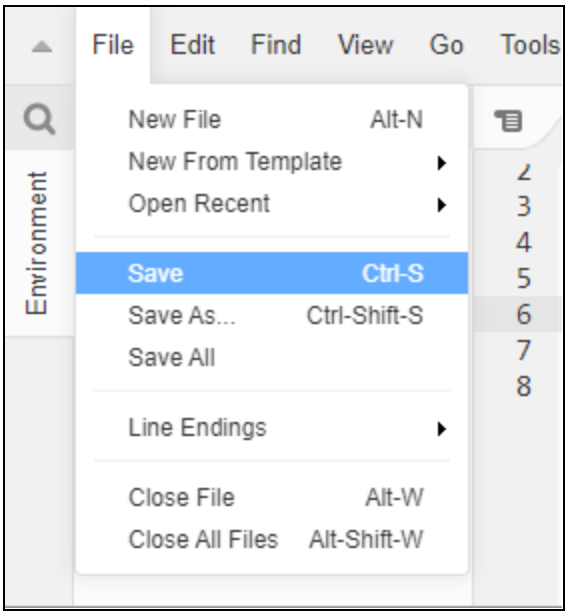


17. Return to the **AWS Lambda console** browser tab, and replace `<REPLACE_WITH_INSTANCE_ID>` with the actual instance ID that you just copied. **Important**: Keep the single quotation marks (' ') around the instance ID in your code. Your code should now look similar to the following example. However, you might have a different value for the Region, and you will have a different value for the instance ID:



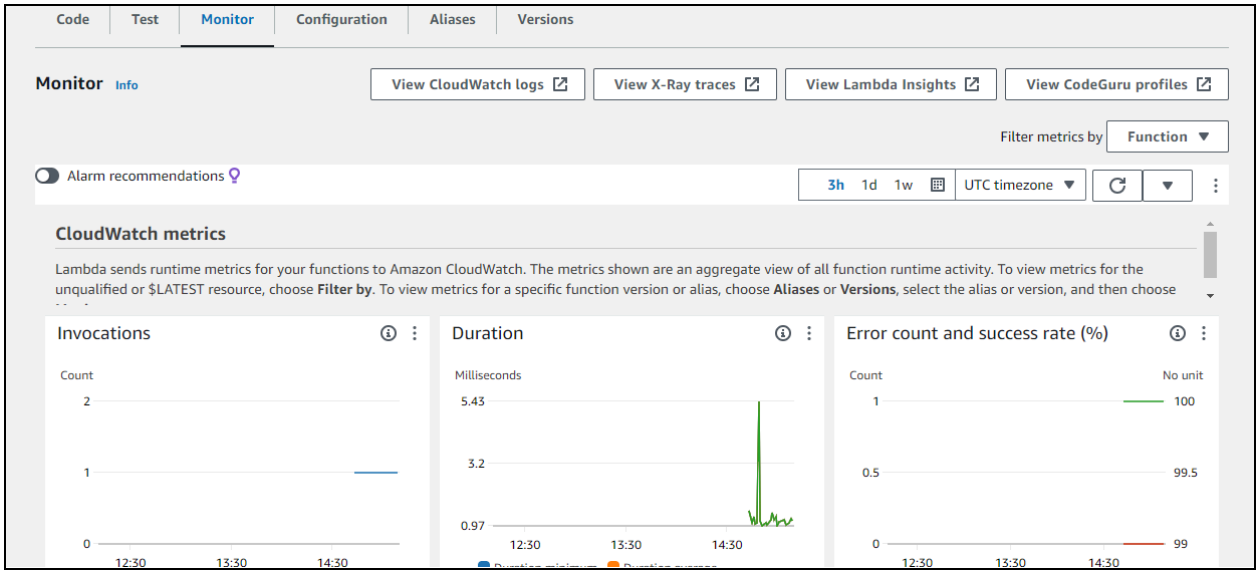
18. Choose the **File** menu and **Save** the changes. Then, in the **Code source** box, choose **Deploy**.

© Copyright upGrad Education Pvt. Ltd. All rights reserved



Your Lambda function is now fully configured. It should attempt to stop your instance every minute.

19. Choose **Monitor** (the tab near the top of the page).
    Note that one of the charts shows you how many times your function has been invoked. There is also a chart that shows the error count and the success rate as a percentage.



Task 4: Verify that the Lambda function worked

19. Return to the **Amazon EC2 console** browser tab and see if your instance was stopped.
    Tip: You can choose the □ refresh icon or refresh the browser page to see the change in state more quickly.

© Copyright upGrad Education Pvt. Ltd. All rights reserved

The EC2 instance named **instance1** disappears from the list of instances, confirming that the Lambda function has worked.

Disclaimer: All content and material on the upGrad website is copyrighted material, either belonging to upGrad or its bonafide contributors and is purely for the dissemination of education. You are permitted to access print and download extracts from this site purely for your own education only and on the following basis:

- You can download this document from the website for self-use only.
- Any copies of this document, in part or full, saved to disc or to any other storage medium may only be used for subsequent, self-viewing purposes or to print an individual extract or copy for non-commercial personal use only.
- Any further dissemination, distribution, reproduction, copying of the content of the document herein or the uploading thereof on other websites or use of the content for any other commercial/unauthorised purposes in any way which could infringe the intellectual property rights of upGrad or its contributors, is strictly prohibited.
- No graphics, images or photographs from any accompanying text in this document will be used separately for unauthorised purposes.
- No material in this document will be modified, adapted or altered in any way.
- No part of this document or upGrad content may be reproduced or stored in any other web site or included in any public or private electronic retrieval system or service without upGrad's prior written permission.
- Any rights not expressly granted in these terms are reserved.

© Copyright upGrad Education Pvt. Ltd. All rights reserved